

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

УДК 004.023

ОТЧЕТ

по исследовательскому курсовому проекту

на тему

**«Оптимизация алгоритмов маршрутизации на примере задачи
нескольких коммивояжеров»**

по направлению подготовки бакалавров 09.03.04 «Программная инженерия»

Выполнил
студент группы БПИ246
Д. Д. Купцов

Научный руководитель
Ермаков Андрей Максимович

Москва 2026

РЕФЕРАТ

В работе представлен исследовательский курсовой проект, в рамках которого изучены и реализованы эвристические подходы к задаче нескольких коммивояжеров (multiple travelling salesman problem, mTSP). Основное внимание уделено варианту с одним депо и основной целевой функцией MINSUM --- минимизацией суммарной длины всех маршрутов. При этом качество решений рассматривается не только через сумму длин, но и через распределение нагрузки между агентами и соблюдение заданного вычислительного бюджета. Объект исследования --- процессы построения маршрутов и распределения вершин между несколькими исполнителями; предмет --- эвристические и метаэвристические алгоритмы решения mTSP при ограниченном вычислительном бюджете.

Цель работы --- спроектировать, реализовать и количественно оценить набор масштабируемых эвристик для построения качественных решений mTSP при ограниченном времени работы, где качество оценивается по суммарной длине маршрутов, сбалансированности нагрузки между агентами и соблюдению временного бюджета. Для её достижения были изучены классические подходы к TSP и mTSP, реализован стек собственных решателей (от простейшего `rand+nn` и `2opt+greed` до модульной архитектуры `src/v21/` с Adaptive Large Neighborhood Search, Simulated Annealing и опциональным Parallel Tempering), подготовлен воспроизводимый экспериментальный контур на C++ и Python, а также проведены сравнения с OR-Tools, преобразованием TSP $\rightarrow$ mTSP и внешними ориентирами LKH-3 и FILO2. Основным итоговым методом рассматривается архитектура `src/v21/`; ранние `lkh-wrapper-v1..v3` и `single-file v12` используются как этапы эволюции этой линии.

Полученные результаты количественно подтверждают эффективность candidate-based локального поиска. На uniform-подвыборке средний относительный отрыв (gap) от OR-Tools+GLS снижается со 114.7% у `rand+nn` до 14.5% у `lkh-wrapper-v2`. На расширенном multifamily-benchmark, включающем 4 семейства синтетических инстансов, 7 алгоритмов, 14 конфигураций (n, m) и 3920 корректных запусков, `lkh-wrapper-v2` оказывается лучше `grasp` в 491 случае из 560 (87.7%) и лучше `2opt+greed` в 558 из 560 (99.6%). Версия `v3` при этом трактуется не как более качественная замена `v2`, а как ускоренная инженерная модификация: она сохраняет близкий MINSUM, но уменьшает среднее время работы. На крупных инстансах ($n \geq 5\,000$) срав-

нение с LKH-3 показывает компромисс: собственная $v12$ на пяти проверенных инстансах быстрее примерно в 6.2 раза и строит более сбалансированные маршруты (отношение $\max/\text{avg}$ --- 1.0--2.7 против 1.7--5.0 у LKH-3), но уступает по MINSUM на $\sim 21\%$. Это мотивировало архитектурное переписывание линии в $v13$ -- $v21$: модульный ALNS-каркас на 22 header-only-модуля показал в variance audit на трёх uniform flagship-инстансах ($n \in \{10000; 50000; 100000\}$, 10 seeds) коэффициент вариации MINSUM 0.17--0.59%; анализ трасс указывает на super-linear-overhead $\approx 2.14\times$ как на вероятное архитектурное ограничение, требующее отдельного профилирования. Для high- m ($m = 100$) был добавлен FILO2-вдохновлённый `lkh_v21_minsum_cap` с capacity-aware repair: на 4 high- m инстансах он даёт парный MINSUM-выигрыш в 3.24% в среднем (16/20 seeds улучшено, Wilcoxon $p = 0.002$), сокращая разрыв до FILO2 на flagship-инстансе с $\sim 30\%$ до $\sim 15.7\%$.

Ключевые слова: задача коммивояжёра, множественный коммивояжёр, mTSP, маршрутизация, эвристические алгоритмы, Lin--Kernighan--Helsgaun, локальный поиск, ALNS, Simulated Annealing, MINSUM, MINMAX, баланс маршрутов, интегральная оценка качества, вычислительный эксперимент.

Содержание

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	6
ВВЕДЕНИЕ	9
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	11
1.1 Место mTSP среди задач TSP и VRP	11
1.2 Формальная модель и варианты постановки	12
1.3 Точные методы и преобразования	14
1.4 Эвристики и метаэвристики	14
1.5 Инженерные приёмы масштабирования	16
1.6 Benchmark-наборы и экспериментальная методология	17
1.7 Выводы для данной работы	18
2 ПОДРОБНЫЙ РАЗБОР ЭВРИСТИК MTSP	18
2.1 Поколения, файлы и границы реконструкции	19
2.2 Базовые baseline: construction, 2-opt и GRASP	20
2.3 Ранняя LKH-стилевая линия v1--v3	22
2.4 Монолитная large-scale версия v12	23
2.5 Флагманская архитектура src/v21/	24
2.6 Специализированные ветки cap и minmax	25
2.7 Внешние ориентиры и литературные соответствия	26
2.8 Ablation-study и инженерные направления	27
3 ПОСТАНОВКА ЗАДАЧИ И РЕАЛИЗАЦИЯ	28
3.1 Формальная постановка в проекте	28
3.2 Формат входа, выхода и CLI-контур	29
3.3 Реализационная архитектура репозитория	30
3.4 Реестр solver-ов и границы собственного вклада	31
3.5 Экспериментальный контур и артефакты	32
3.6 Модульная реализация src/v21/	32
3.7 Особенности LKH-inspired реализации	33
4 ЛИНИЯ СОБСТВЕННЫХ LKH-ПОДОБНЫХ АЛГОРИТМОВ	34
4.1 Ранняя линия v1..v3	34

4.2	Масштабируемая single-file версия v12	35
4.3	Итоговая архитектура src/v21/	36
4.4	Специализированные ветки car и minmax	36
5	МЕТОДИКА ВЫЧИСЛИТЕЛЬНОГО ЭКСПЕРИМЕНТА	37
5.1	Аппаратная конфигурация и сборка	37
5.2	Семейства тестовых инстансов	37
5.3	Метрики	38
5.4	Валидация и воспроизведение	39
6	РЕЗУЛЬТАТЫ И ИХ АНАЛИЗ	40
6.1	Базовое сравнение на uniform-инстансах	40
6.2	Сравнение с OR-Tools на uniform	41
6.3	Расширенный multifamily-benchmark	41
6.4	Попарные сравнения на идентичных инстансах	42
6.5	Стабильность solver-ов: пилотный анализ cv	44
6.6	Сравнение v12 с внешним LKH-3 на $n = 5000$	44
6.7	Архитектура src/v21/: ALNS + SA + Parallel Tempering	46
6.8	Variance audit и диагностика flagship-инстансов	49
6.9	FIL02-вдохновлённая стабилизация для high-m MINSUM	51
6.10	Anytime-профиль на flagship-инстансе	55
6.11	Чувствительность к числу агентов m	56
6.12	MINMAX-режим: lkh_v21_minmax	57
6.13	Интегральная оценка IQS на крупном uniform-инстансе	59
6.14	Интерпретация результатов	60
6.15	Что доказано проведёнными экспериментами	61
7	ДОСТОВЕРНОСТЬ, ОГРАНИЧЕНИЯ И РИСКИ	62
	ЗАКЛЮЧЕНИЕ	63
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	67
	ПРИЛОЖЕНИЕ А. РЕКОМЕНДУЕМЫЙ СОСТАВ ПОЛНОЙ ТАБ-	
	ЛИЦЫ ЭКСПЕРИМЕНТОВ	71

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Термин	Определение
TSP	Travelling Salesman Problem --- задача о коммивояжёре. Требуется найти замкнутый обход всех вершин, проходящий через каждую ровно один раз и имеющий наименьшую суммарную длину.
mTSP	Multiple Travelling Salesman Problem. Обобщение TSP на m исполнителей, стартующих из общего депо: каждая клиентская вершина должна быть посещена ровно одним маршрутом.
Депо	Выделенная вершина, в которой все маршруты начинаются и в которую возвращаются.
Агент, коммивояжёр	Один из m исполнителей, которому соответствует отдельный маршрут в решении mTSP.
MINSUM	Целевая функция, равная сумме длин всех m маршрутов решения. Меньшее значение соответствует лучшему решению.
MINMAX	Альтернативная целевая функция: минимизация длины самого длинного маршрута. В работе используется как контекст для интерпретации баланса маршрутов, а не как основной критерий.
imbalance	Относительный показатель несбалансированности маршрутов: отношение длины самого длинного маршрута к средней длине маршрута, $m \cdot \text{MINMAX} / \text{MINSUM}$. Значение 1 соответствует идеально равным по длине маршрутам.
IQS	Integrated Quality Score --- интегральная оценка качества решения, объединяющая близость MINSUM к лучшему найденному значению, баланс маршрутов и штраф за превышение временного бюджета.

Термин	Определение
2-opt	Локальное преобразование тура: удаляются два неинцидентных ребра, между ними переворачивается фрагмент маршрута; ход применяется при уменьшении длины.
k -opt	Обобщение 2-opt с заменой k рёбер.
LKH	Lin--Kernighan--Helsgaun --- семейство сильных эвристик для TSP, основанных на локальном поиске переменной глубины и кандидатных рёбрах.
Candidate set	Ограниченное множество перспективных соседей для каждой вершины. Используется для замены полного перебора $O(n^2)$ пар вершин на $O(n \cdot k)$ ходов с $k \ll n$.
GRASP	Greedy Randomized Adaptive Search Procedure: рандомизированная многостартовая процедура, чередующая жадное конструирование (с RCL) и улучшение.
RCL	Restricted Candidate List --- ограниченный список лучших локальных кандидатов внутри GRASP.
ALNS	Adaptive Large Neighborhood Search --- метаэвристический каркас, чередующий destroy/repair-операторы с адаптивным выбором их веса.
SA	Simulated Annealing --- стохастическое правило приёма ходов, основанное на функции Метрополиса с убывающей температурой.
PT	Parallel Tempering --- ансамблевый поиск с обменом конфигураций между репликами разных температур.
OR-Tools	Open-source-библиотека Google для задач оптимизации; используется как внешний эвристический ориентир.

Термин	Определение
FILO2	Сильный эвристический CVRP-решатель, использующийся в работе как внешний ориентир по идеям масштабирования и для high- m -сравнений.
Benchmark	Семейство тестовых инстансов, на которых сравниваются алгоритмы.
Relative gap	Относительное отклонение от ориентира: $100 \cdot (A - B)/B$, где A --- значение исследуемого алгоритма, B --- ориентира.
cv	Коэффициент вариации σ/μ , обычно в процентах. Используется как метрика стабильности эвристики.

ВВЕДЕНИЕ

Маршрутизация является одной из наиболее прикладных областей комбинаторной оптимизации. Она возникает в логистике, транспортном планировании, сервисном обслуживании и планировании производственных операций, то есть в задачах, где требуется упорядочить перемещения по множеству точек и распределить их между исполнителями. Математической основой многих таких постановок служит задача коммивояжёра: необходимо построить замкнутый маршрут минимальной длины, проходящий через все вершины ровно один раз [1,2].

TSP относится к классу NP-трудных задач, поэтому при росте числа вершин точные методы и полный перебор становятся вычислительно непрактичными. Во многих прикладных сценариях вместо одного исполнителя используется несколько агентов, стартующих из общего депо и совместно обслуживающих набор клиентов. Такая постановка известна как задача нескольких коммивояжёров, или mTSP. По сравнению с классической TSP в ней появляется дополнительный уровень сложности: необходимо не только определить порядок обхода вершин, но и распределить клиентские вершины между агентами [4].

В данной работе рассматривается mTSP с одним депо и основной целевой функцией MINSUM --- минимизацией суммарной длины всех маршрутов. Однако одного MINSUM недостаточно для полной прикладной интерпретации решения: при близкой суммарной длине один агент может получить существенно более длинный маршрут, чем остальные. Поэтому в экспериментальной части дополнительно учитывается баланс нагрузки через показатель *imbalance* и вводится интегральная оценка качества IQS, в которой MINSUM остаётся главным компонентом, а баланс и время работы выступают дополнительными факторами. Получение точного оптимума на инстансах средней и большой размерности обычно вычислительно непрактично, поэтому основной интерес представляет построение качественного приближённого решения в рамках заданного вычислительного бюджета.

Актуальность исследования определяется двумя обстоятельствами. Во-первых, mTSP служит ядром для более сложных постановок семейства Vehicle Routing Problem (VRP); современный обзор [35] перечисляет десятки прикладных направлений, в которых mTSP используется как компонент. Во-вторых, во многих современных эвристиках качество и масштабируемость достигаются не

прямым расширением пространства поиска, а сочетанием нескольких инженерных приёмов: ограниченных множеств кандидатных рёбер (candidate edges), локальных перестановок, кластеризации, разреженных соседств, декомпозиции, кэширования и повторных улучшений [3,7,8,10,28,29]. Поэтому практический интерес представляет не только выбор готовой библиотеки, но и анализ решений, которые делают эвристику управляемой и масштабируемой.

Объект исследования --- процесс построения маршрутов для нескольких агентов в задачах комбинаторной оптимизации. **Предмет исследования** -- эвристические и метаэвристические алгоритмы распределения клиентских вершин между агентами и оптимизации порядка их обхода в mTSP.

Цель работы --- спроектировать, реализовать и экспериментально оценить эвристические методы построения качественных решений для mTSP с одним депо при фиксированном вычислительном бюджете, где качество решения оценивается по суммарной длине маршрутов, сбалансированности нагрузки между агентами, времени работы, стабильности и масштабируемости. Для достижения цели поставлены следующие задачи:

1. изучить постановку TSP и mTSP, сопутствующие целевые функции и ограничения;
2. проанализировать существующие подходы --- локальный поиск, GRASP, LKH, candidate sets, ALNS;
3. реализовать единый воспроизводимый экспериментальный контур для запуска алгоритмов и независимой проверки решений;
4. подготовить семейства синтетических mTSP-инстансов с разными геометрическими свойствами;
5. провести сравнения базовых эвристик, собственных LKH-стилевых версий и внешних ориентиров по MINSUM и дополнительным метрикам баланса;
6. оценить устойчивость стохастических алгоритмов к изменению seed и провести парные статистические сравнения там, где это допускает дизайн эксперимента;
7. проанализировать применимость разработанных подходов при разных размерах инстансов, числе агентов и типах геометрии, а также выявить ограничения текущей реализации.

Рабочая гипотеза. Эвристики, использующие ограниченные множества

кандидатных рёбер, локальный поиск и LKH-подобные процедуры улучшения, позволяют получать более качественные решения mTSP по MINSUM, чем простые конструктивные алгоритмы, при контролируемом вычислительном бюджете. Дополнительно проверяется инженерная гипотеза о том, что собственные LKH-стилевые версии могут быть полезны в режимах жёсткого ограничения времени и давать более сбалансированные маршруты, даже если по чистому MINSUM уступают полноформатным внешним решателям, таким как LKH-3 или FILO2.

Методы. В работе использованы анализ литературы, формализация задачи, реализация решателей на C++17, вычислительные эксперименты с независимой валидацией маршрутов и агрегирование результатов в едином контуре запуска. Для части парных сравнений применялся непараметрический критерий Wilcoxon signed-rank; устойчивость стохастических запусков дополнительно оценивалась через стандартное отклонение и коэффициент вариации. Эксперименты проводились на синтетических инстансах с разными распределениями: равномерным, кластерным, кластерным со смещённым депо, смешанным с выбросами и стрессовым с большим числом агентов.

Эксперименты проводились в едином воспроизводимом контуре запуска на одной основной аппаратной конфигурации; подробности сборки, параметров и ограничений приведены в разделе методики.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Место mTSP среди задач TSP и VRP

Классическая задача коммивояжёра (Travelling Salesman Problem, TSP) является базовой моделью маршрутизации: для заданного множества вершин и функции расстояния требуется построить замкнутый маршрут минимальной длины, проходящий через каждую вершину ровно один раз. В графовой форме пусть $V = \{0, 1, \dots, n - 1\}$, а $d(i, j)$ задаёт стоимость перехода между вершинами. Тогда TSP ищет перестановку вершин, минимизирующую

$$\min_{\pi} \left(d(\pi_n, \pi_1) + \sum_{i=1}^{n-1} d(\pi_i, \pi_{i+1}) \right). \quad (1)$$

В евклидовом симметричном случае вершины задаются координатами на плоскости, $d(i, j) = d(j, i)$, а расстояние вычисляется как евклидова норма. Именно

такая геометрическая постановка часто используется в вычислительных экспериментах, потому что она сохраняет прикладную интерпретацию маршрутизации и одновременно позволяет контролируемо генерировать данные.

Задача нескольких коммивояжёров (multiple Travelling Salesman Problem, mTSP) расширяет TSP: вместо одного маршрута строится набор маршрутов для m агентов. Обычно агенты стартуют из общего депо и возвращаются в него, а каждая клиентская вершина должна быть обслужена ровно один раз. В этом смысле mTSP занимает промежуточное положение между TSP и задачами маршрутизации транспорта (Vehicle Routing Problem, VRP): как и TSP, она описывает порядок обхода вершин, но как и VRP, требует распределить клиентов между несколькими маршрутами. Обзоры Bektaş [4] и Cheikhrouhou-Khoufi [35] подчёркивают, что небольшие изменения постановки mTSP существенно влияют на сравнимость алгоритмов: число депо, число используемых маршрутов, разрешение пустых маршрутов и выбор целевой функции меняют допустимую область решений.

1.2 Формальная модель и варианты постановки

Для single-depot closed mTSP решение можно представить набором маршрутов

$$R_s = (0, v_1^s, v_2^s, \dots, v_{k_s}^s, 0), \quad s = 1, \dots, m, \quad (2)$$

где вершина 0 является депо, а все клиентские вершины из $V \setminus \{0\}$ распределяются между маршрутами без повторов. Основная целевая функция MINSUM имеет вид

$$F_{\text{sum}}(R_1, \dots, R_m) = \sum_{s=1}^m \sum_{i=0}^{k_s} d(v_i^s, v_{i+1}^s), \quad v_0^s = v_{k_s+1}^s = 0. \quad (3)$$

Альтернативная цель MINMAX минимизирует длину самого длинного маршрута:

$$F_{\text{max}}(R_1, \dots, R_m) = \max_{s=1, \dots, m} \text{len}(R_s). \quad (4)$$

MINSUM соответствует минимизации суммарной стоимости, топлива или общего времени обслуживания, тогда как MINMAX ближе к задачам равномерной загрузки исполнителей и ограничения максимального времени работы. Поэтому баланс маршрутов не заменяет MINSUM-цель и должен учитываться явно:

либо через отдельную многокритериальную постановку, либо через дополнительную интегральную оценку, в которой веса компонентов фиксируются заранее [20,23]. В данной работе такая оценка используется только как экспериментальный агрегат для интерпретации качества решений; оптимизационная цель большинства solver-ов остаётся MINSUM.

В данной работе основной рассматриваемый вариант фиксируется следующим образом: одно депо, замкнутые маршруты, симметрическая евклидова метрика, заданное число m маршрутных записей и основная цель MINSUM. На уровне экспериментального валидатора допускается вырожденный маршрут депо--депо как технический случай, однако в основных наборах $n - 1 \gg m$, поэтому маршруты фактически непустые. Если внешний baseline использует *at-most-m* трактовку, CVRP-адаптацию или ограничения вместимости, такое сравнение интерпретируется как сравнение со смежным ориентиром, а не как строго тождественное сравнение одной и той же допустимой области.

Вариант	Смысл	Почему важно для сравнения
<i>exactly-m</i>	решение содержит ровно m маршрутов	фиксирует число агентов; отличается от постановок, где часть агентов может не использоваться
<i>at-most-m</i>	разрешено использовать не больше m маршрутов	может давать меньший MINSUM за счёт отказа от части маршрутов
nonempty routes	каждый маршрут содержит хотя бы одного клиента	меняет интерпретацию баланса и числа исполнителей
single depot / multi-depot	одно или несколько депо	меняет геометрию допустимых маршрутов
closed / open routes	возврат в депо обязателен или нет	напрямую меняет длины маршрутов и сравнимость MINSUM

Вариант	Смысл	Почему важно для сравнения
MINSUM / MINMAX	минимизация суммы или максимума маршрутов	отражает разные прикладные цели: стоимость против баланса

1.3 Точные методы и преобразования

TSP и mTSP относятся к NP-трудным задачам, поэтому с ростом числа вершин точные методы быстро становятся вычислительно дорогими. Тем не менее exact-line важна для обзора предметной области: она задаёт формальные модели, малые эталоны качества и язык, на котором затем описываются приближённые алгоритмы. Классическая линия TSP включает постановку Dantzig--Fulkerson--Johnson с отсечениями подтуров [37], компактную MTZ-формулировку [38], динамическое программирование Held--Karp [39] и дальнейшее развитие branch-and-cut-подхода.

Для mTSP используются как прямые целочисленные формулировки, так и преобразования к TSP-подобным моделям. Kara и Bektaş [40] рассматривают ILP-формулировки mTSP и его вариантов, а Jonker--Volgenant [5] предлагают преобразование симметрического mTSP к TSP. Такие преобразования полезны тем, что позволяют использовать сильные TSP-решатели, но требуют аккуратной настройки: копии депо, штрафные функции, число маршрутов и ограничения непустоты должны соответствовать выбранной постановке. Поэтому внешний TSP/LKH-подобный baseline в отчёте рассматривается как сильный ориентир, но его формальная сопоставимость зависит от конкретного адаптера.

1.4 Эвристики и метаэвристики

На средних и больших инстансах основную практическую роль играют эвристики. Простые конструктивные методы строят решение последовательно: nearest neighbor, insertion-based эвристики и жадное распределение клиентов дают быстрый старт, но закрепляют ранние локальные ошибки. Локальный поиск 2-opt и 3-opt [6] улучшает порядок обхода внутри маршрута, устраняя неудачные рёбра и пересечения, однако сам по себе не решает задачу перераспределения клиентов между агентами. Для mTSP поэтому важны не только *intra-route* ходы, но и *inter-route* операции: relocate, swap, cross-exchange и обмены цепоч-

ками.

Lin--Kernighan-подход [1] усиливает локальный поиск за счёт переменной глубины: вместо фиксированного 2-opt или 3-opt алгоритм строит последовательность замен рёбер. LKH Helsgaun [2] делает эту линию практически сильной благодаря candidate sets, alpha-nearness и правилам управления поиском. LKH-3 [3] расширяет LK/LKH-подход на ряд constrained TSP/VRP-постановок; для mTSP-подобных задач используются problem-specific transformations и penalty mechanisms, которые необходимо оговаривать относительно выбранной формализации.

Метаэвристики добавляют механизмы диверсификации и выхода из локальных минимумов. GRASP [9] сочетает randomized construction через restricted candidate list и локальное улучшение. VNS для mTSP развивает идею смены окрестностей [42]: если одна окрестность перестаёт улучшать решение, поиск переходит к другой. Генетические и меметические методы используют популяции решений, кроссовер и интенсивное локальное улучшение; для mTSP и MINMAX-mTSP эта линия представлена, например, работами Brown--Ragsdale--Carter [43] и He--Hao [31,44]. В VRP-литературе особенно важны ALNS [24,25], HGS [26,27] и SISR [28]: они показывают, что сильный routing-солвер обычно является гибридом construction, local search, destroy/repair и специализированных структур данных.

Класс методов	Основная идея	Сильные стороны	Ограничения
Точные методы	ILP, DP, branch-and-cut	оптимум и сертификат качества	плохо масштабируются
Конструктивные эвристики	быстрое построение начального решения	минимальное время, удобны как baseline	миопичность решений
2-opt/3-opt	локальная замена рёбер	простой и сильный слой улучшения	без inter-route ходов не меняют назначение клиентов

Класс методов	Основная идея	Сильные стороны	Ограничения
LK/LKH	локальный поиск переменной глубины с кандидатными рёбрами	высокий уровень качества на TSP-like постановках	сложная настройка и адаптация к ограничениям
GRASP/VNS	multi-start и смена окрестностей	выход из части локальных минимумов	чувствительность к бюджету и параметрам
ALNS/HGS/SISR destroy/repair, population search, intensive education		сильные современные VRP-каркасы	выше сложность реализации и настройки

1.5 Инженерные приёмы масштабирования

Для больших n хранение полной матрицы расстояний и просмотр всех пар вершин становятся дорогостоящими по памяти и времени. Поэтому современные routing-эвристики переходят к разреженным и локализованным представлениям: candidate lists, granular neighborhoods, spatial indexing, lazy distance cache, delta-evaluation и selective recomputation. В LKH-family candidate sets ограничивают множество рёбер, которые реально рассматриваются в локальном поиске [2,7]; в granular tabu search Toth--Vigo [29] из окрестности исключаются заведомо длинные рёбра; в крупномасштабных VRP-эвристиках используются локальные области изменения и маршрутоориентированные структуры данных [8,30].

Важно, что масштабирование не сводится только к candidate sets. Для mTSP критичны также структуры маршрута и стоимость применения хода: 2-opt может требовать переворота длинного фрагмента, relocate и swap затрагивают несколько маршрутов, repair-фаза должна быстро оценивать вставку клиента в множество позиций. Поэтому сильные реализации используют delta-evaluation, массивы позиций, successor/predecessor-представления, списки затронутых вершин, пространственные индексы и иногда декомпозицию крупной задачи на локальные зоны. Эти приёмы не меняют математическую постановку

ку, но определяют, можно ли вообще получить полезное решение в заданном time-budget.

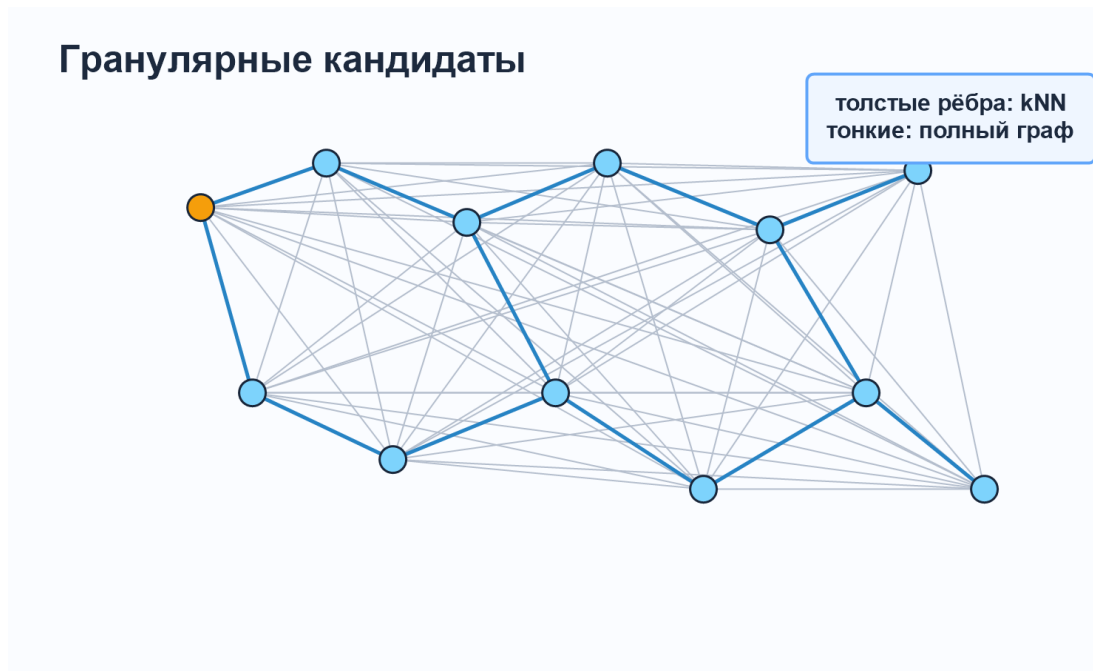


Рис. 1: Принцип гранулярных кандидатов: вместо полного графа K_n с $O(n^2)$ рёбрами для каждой вершины хранится ограниченное число перспективных соседей.

1.6 Benchmark-наборы и экспериментальная методология

Обзор предметной области должен учитывать не только алгоритмы, но и культуру сравнения. Для TSP классическим публичным набором является TSPLIB [13]; для VRP и геометрически разнообразных routing-инстансов широко используются CVRPLIB и X-instances Uchoa и соавт. [41]. Синтетические семейства также полезны: uniform, clustered, shifted-depot, mixed-outliers и high- m stress позволяют изолированно проверять влияние геометрии, числа агентов и распределения клиентов. Однако синтетика не заменяет публичные benchmark-наборы, а дополняет их.

Честный экспериментальный протокол для эвристик должен явно фиксировать допустимую область, time-budget, random seed schedule, thread policy, метрики и критерии валидности. Основная метрика в данной работе --- MINSUM; вторичные метрики включают runtime, valid-rate, длину самого длинного маршрута, imbalance ratio, coefficient of variation по seed и wins/ties/losses в попарных сравнениях. Для стохастических алгоритмов полезны не только

средние значения, но и median/IQR, Wilcoxon signed-rank для парных разностей, coefficient of variation и anytime-кривые quality-vs-time [14,15,32,33,34]. Если сравнивается mTSP-решатель с CVRP/VRP-related baseline, в тексте должна быть отдельная оговорка о различии feasible region.

1.7 Выводы для данной работы

Из литературы следует несколько требований к итоговой архитектуре отчёта и эксперимента. Во-первых, обзор предметной области должен отделяться от описания собственной реализации: в первой главе уместно говорить о классах методов, а конкретные имена внутренних решателей должны появляться в главах реализации и результатов. Во-вторых, основной вариант задачи необходимо фиксировать явно: single-depot, closed, symmetric Euclidean, MINSUM как primary objective, а баланс --- как secondary diagnostic. В-третьих, качество эвристики следует оценивать не одной итоговой таблицей, а сочетанием валидности, MINSUM, runtime, устойчивости по seed и парных сравнений.

Именно эта логика используется далее в отчёте. Сначала подробно разбираются классы реализованных эвристик и внешние ориентиры, затем формализуется проектная постановка, после этого описывается эволюция собственных LKH-подобных алгоритмов и только в экспериментальной главе обсуждаются численные результаты. Такой порядок позволяет не смешивать обзор литературы, архитектуру кода и интерпретацию полученных данных.

2 ПОДРОБНЫЙ РАЗБОР ЭВРИСТИК MTSP

Эта глава связывает обзор предметной области с конкретной реализацией проекта. В отличие от первой главы, здесь уже уместны имена solver-ов, файлов и внутренних модулей: задача главы --- показать, какие эвристические идеи фактически реализованы, как они эволюционировали и почему разные версии нельзя оценивать по одной линейной шкале <<лучше/хуже>>.

Линия решателей распадается на два поколения. Первое поколение -- lkh-wrapper-v1..v12: собственные LKH-стилевые эвристики, где основной выигрыш достигается за счёт candidate sets, локального поиска, дешёвых perturbation-приёмов и межмаршрутных обменов. Второе поколение --- src/v21/ и его ветки cap/minmax: модульная метаэвристическая архитектура с constructive phase, ALNS, Simulated Annealing, optional Parallel Tempering, GLS, POPMUSIC-style zone moves и разными accept-policy для MINSUM и

MINMAX. Поэтому корректная интерпретация такая: v3 --- финальная ранняя версия, v12 --- завершающий монолитный large-scale milestone, а v21 --- основной архитектурный результат работы.

Основная цель работы --- MINSUM. Баланс маршрутов используется как вторичная диагностическая характеристика, а не как часть MINSUM-целевой функции. Поэтому далее для каждого алгоритма отдельно обсуждаются: вклад в MINSUM, скорость, масштабируемость и влияние на распределение клиентов между маршрутами.

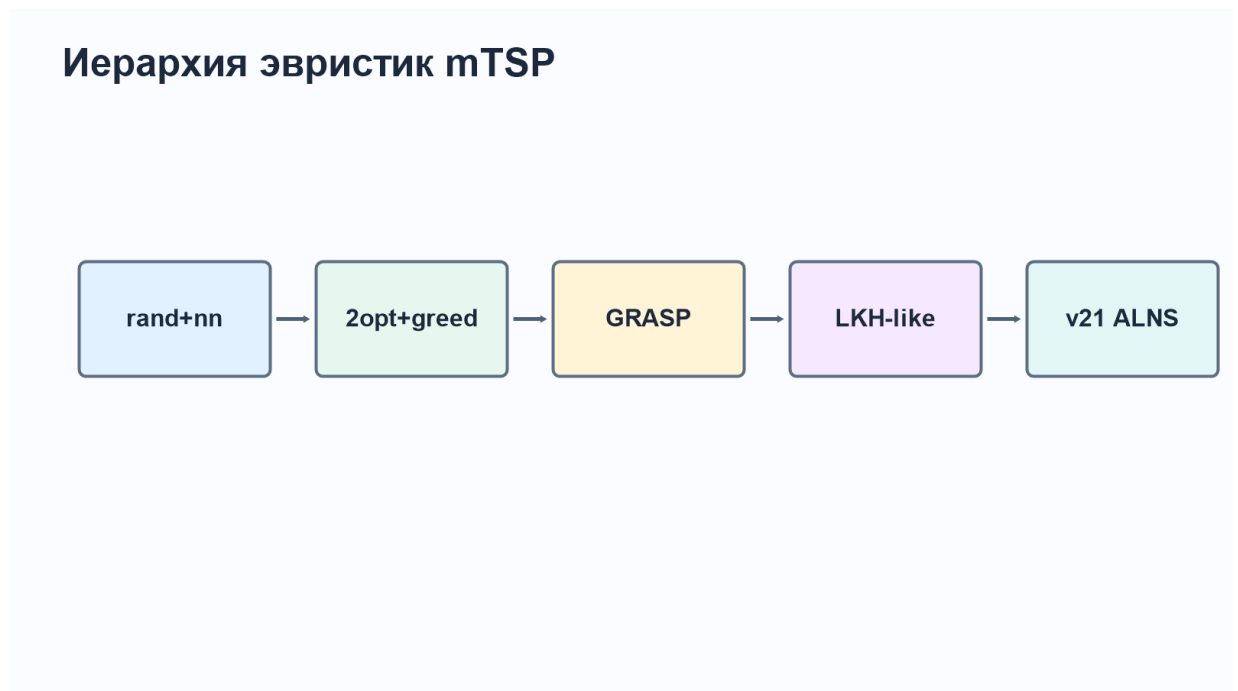


Рис. 2: Эволюция реализованных эвристик: от простых baseline через ранние LKH-стилевые версии к монолитной large-scale версии v12 и флагманской архитектуре v21.

2.1 Поколения, файлы и границы реконструкции

Для ранних версий v1--v3 разбор опирается на конкретные исходные файлы `src/mtsp_lkh_wrapper_v1.cpp`, `src/mtsp_lkh_wrapper_v2.cpp` и `src/mtsp_lkh_wrapper_v3.cpp`. Поэтому для них можно говорить на уровне отдельных функций. Для v12 и v21 исходники также присутствуют, но их объём заметно больше; в отчёте они описываются на уровне архитектурных блоков и явно идентифицированных модулей, без попытки построчно пересказывать весь код.

Файл / версия	Ключевые функции и модули	Эвристическая роль
mtsp_lkh_wrapper_v1.cpp	ComputeLengthGenericV1, ImproveRoute2OptGenericV1, DoubleBridgeKickV1, IteratedLocalSearchV1, ImproveInterRoute	базовый LKH-style wrapper: greedy construction, полный intra-route 2-opt, double-bridge kick, ILS и exhaustive межмаршрутный swap
mtsp_lkh_wrapper_v2.cpp	BuildCandidateSetsV2, ForwardPotentialV2, ApplyBestPositiveGain2OptV2, ImproveRouteGuidedV2, BuildInitialRoutes	первый полноценный candidate-based solver: k-NN кандидаты, lookahead-aware construction, depot-aware score, candidate-guided 2-opt
mtsp_lkh_wrapper_v3.cpp	FindCandidateSet, IsPromisingSwapPair, SwapDeltaClosedRoutes, ApplyFirstImproving2OptV3, dont-look bits	инженерная оптимизация v2: first-improvement, delta-evaluation для swap, фильтрация межмаршрутных пар, ускорение без заявки на лучшее среднее качество
mtsp_lkh_wrapper_v12.cpp	geometric candidates, lightweight clusters, savings seed, cluster-aware construction, rebalance, late repair, route annealing	финальный монолитный large-scale milestone: быстрый первый ответ, отдельный budget на улучшение, cluster/savings-aware старт и MINSUM-архив
src/v21/core/00--21	DistanceOracle, KDTree2D, CandidateSet, RouteList, seed portfolio, local search, destroy/repair, SA, PT, autotune, GLS, route-pair reopt, POPMUSIC	модульная архитектура второго поколения: общий pipeline с plug-in accept-policy для MINSUM/MINMAX и статистикой операторов
minsum_solver.cpp, minsum_solver_cap.cpp, minmax_solver.cpp	entry points и accept-policy	три внешних режима одного архитектурного ядра: MINSUM, high- m cap-stabilization и MINMAX

Эта карта показывает структурный перелом: ранние версии --- monolithic single-file solvers, где construction, local search, perturbation и acceptance слиты в одном файле. v21 разводит эти уровни по модулям, что ближе к современным VRP-каркасам: сильный solver становится не одной эвристикой, а композицией seed-портфеля, локального поиска, destroy/repair-операторов, accept-policy и инженерных ограничителей окрестности.

2.2 Базовые baseline: construction, 2-opt и GRASP

Решатель `mtsp_rand_nn_solver`, зарегистрированный как `rand+nn`, выполняет две фазы: случайное распределение клиентов между m маршрутами с фиксируемым seed и nearest-neighbor-достраивание внутри каждой группы. Его роль --- не конкурировать с сильными эвристиками, а задавать нижний уровень качества и проверять экспериментальный контур. Если сложный solver проигрывает `rand+nn`, это почти всегда указывает на ошибку параметров или реализации.

`mtsp_greedy_2opt_solver` (2opt+greed) усиливает базовую схему локальным поиском: после жадного round-robin-построения каждый маршрут независимо улучшается 2-opt. Такой слой устраняет пересечения и неудачные участки внутри маршрута, но почти не меняет распределение клиентов

между агентами. Поэтому 2opt+greed полезен как baseline вклада intra-route local search, но не как полноценный mTSP-solver.

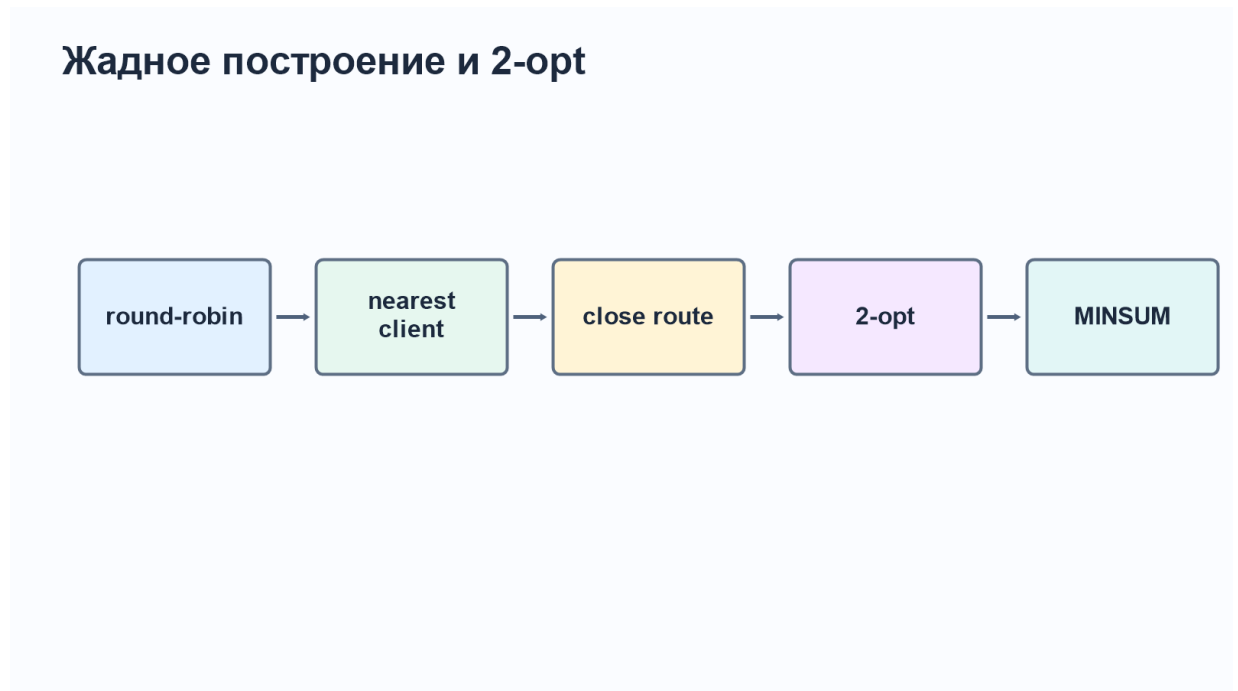


Рис. 3: Логика 2opt+greed: жадное распределение клиентов, замыкание маршрутов в депо и независимое 2-opt-улучшение каждого маршрута.

`mtsp_grasp_solver` реализует GRASP-схему [9]: randomized construction через restricted candidate list, локальный 2-opt и межмаршрутные обмены. Параметры `iters`, `rcl` и `seed` управляют числом рестартов, уровнем жадности и воспроизводимостью. Главный плюс GRASP --- способность частично исправлять ошибки начального назначения за счёт multistart и swap-фазы; главный минус --- рост времени почти линейно с числом стартов и чувствительность к параметрам.

GRASP и restricted candidate list

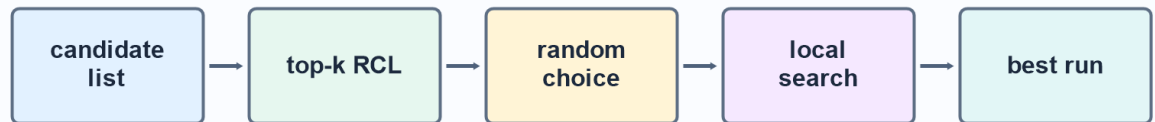


Рис. 4: Restricted candidate list: следующая вершина выбирается случайно из $\text{top-}k$ ближайших, а не безусловно из ближайшей.

2.3 Ранняя LKH-стилевая линия v1--v3

Версия v1 является первой собственной LKH-стилевой обёрткой, но по сути ещё близка к классическому локальному поиску. Она строит начальные маршруты, применяет полный 2-opt внутри каждого маршрута, использует double-bridge kick для выхода из локального минимума и затем запускает Iterated Local Search по маршрутам. Межмаршрутная фаза в v1 полезна концептуально, но дорога вычислительно: swap-кандидаты фактически проверяются с полной переоценкой objective, что плохо масштабируется.

Главный алгоритмический скачок происходит в v2. Здесь появляются candidate sets, lookahead-aware construction, depot-aware score и candidate-guided 2-opt. Это сближает собственный solver с логикой LK/LKH: вместо просмотра всех возможных рёбер алгоритм концентрируется на малом наборе геометрически перспективных соседей. В экспериментальной части именно v2 показывает лучший средний MINSUM среди ранних версий на малых и средних тестах.

v3 не следует описывать как самую качественную версию v1--v3. Её вклад другой: first-improvement вместо более дорогого поиска лучшего хода, dont-look bits, фильтр promising-pairs и delta-evaluation межмаршрутного swap. Поэтому v3 является финальной инженерной модификацией ранней линии: она

почти сохраняет качество v2, но уменьшает среднее время работы. В терминах отчёта это лучший компромисс скорость/качество, тогда как по среднему MINSUM ранний ориентир остаётся за v2.

Версия	Ключевое изменение	Как интерпретировать результат
v1	full 2-opt, double-bridge, ILS, exhaustive inter-route swap	доказательство работоспособности LKH-style идеи, но без достаточного ограничения окрестности
v2	candidate sets, lookahead, candidate-guided 2-opt	основной качественный скачок ранней линии; лучшая по среднему MINSUM на рассмотренных small/mid тестах
v3	first-improvement, dont-look bits, delta swap, promising-pair filter	ускоренная инженерная версия v2; лучше формулировать как speed/quality trade-off

2.4 Монолитная large-scale версия v12

lkh-wrapper-v12 завершает монолитную ветку v1--v12. В отличие от v1--v3, она уже ориентирована на крупные инстансы и быстрый возврат валидного решения. В коде явно разделены budget на получение первого решения и budget на улучшение; строятся geometric candidate sets, используются lightweight clusters, savings-seed, cluster-aware construction, rebalance, late repair и route annealing. Отдельно поддерживается MINSUM-archive: даже если балансирующая фаза улучшает max-route, solver должен помнить лучшее найденное решение по основной MINSUM-цели.

Поздняя схема lkh-wrapper



Рис. 5: Обобщённая схема v12: candidate sets, fast seed, savings/cluster-aware construction, intra-route ILS, inter-route moves, repair и общий time-budget.

Научно аккуратная формулировка для v12: это не доказательство превосходства над LKH-3, а быстрый собственный milestone. В head-to-head на пяти инстансах $n = 5000, m = 5$ он был примерно в 6.2 раза быстрее LKH-3, но уступал по MINSUM примерно на 21 %. Поэтому v12 полезна как инженерный шаг к масштабируемости и как база для дальнейшего архитектурного переписывания, но не как финальный quality-oriented solver.

2.5 Флагманская архитектура src/v21/

src/v21/ является вторым поколением проекта. В ней LKH-style local search превращён в часть более широкого ALNS/SA/PT pipeline. Верхний уровень 17_pipeline.hpp выполняет: построение k-NN candidate sets через KDTree2D, seed portfolio, параллельный polish, ALNS-loop с destroy/repair-операторами, SA-acceptance, optional Parallel Tempering, GLS edge penalties, route-pair reoptimization, POPMUSIC-style step и финальный polish. 18_autotune.hpp задаёт параметры по размеру задачи и цели: k_{NN} , destroy-size, число ILS-rounds, PT replicas, GLS и POPMUSIC-period.

Блок v21	Назначение
02_distance.hpp,	distance oracle, spatial index и candidate graph; основа
03_kdtree.hpp,	масштабируемого просмотра окрестности
06_candidate_set.hpp	
05_route_list.hpp	представление маршрутов, длины маршрутов, dirty-флаги, RouteSize; используется также в cap-ветке
07_seed_routes.hpp	портфель начальных решений: round-robin NN, fast NN, polar sweep, balanced k-means, savings-like construction
08--10	intra-route 2-opt/3-opt-light и inter-route moves; локальное улучшение до и после ALNS
12--14	ALNS framework, destroy-операторы и repair-операторы Cheapest/Regret2; в cap-ветке добавляется capacity-aware skip/fallback
15--17	Simulated Annealing, optional Parallel Tempering и общий orchestration pipeline
18--21	autotune, GLS penalties, route-pair reoptimization, POPMUSIC-style zone decomposition
minsum_accept.hpp	plug-in accept-policy: чистый MINSUM или soft- α
minmax_accept.hpp	MINMAX/MINSUM blend

Именно v21, а не v3 или v12, логично считать главным итоговым методом работы. v3 отвечает на вопрос, сколько даёт candidate-based локальный поиск на малых/средних инстансах; v12 отвечает на вопрос, как далеко можно продвинуть монолитный solver; v21 отвечает на вопрос, как построить расширяемую архитектуру с несколькими целевыми режимами, операторной статистикой, anytime-trace и специализациями для high- m .

2.6 Специализированные ветки cap и minmax

lkh_v21_minsum_cap следует называть именно FILO2-inspired capacity-aware repair: это точечное заимствование инженерного принципа, а не воспроизведение FILO2. Из FILO2 заимствуется не весь solver, а принцип: дешёвое ограничение допустимости repair/move может сильно изменить траекторию поиска. В коде это выражено через route_cap, cap-skip в repair-операторах и cap-aware fallback. Эксперименты показывают пользу этой идеи главным образом в high- m режимах; для малых и средних m её нельзя считать универсальным улучшением.

lkh_v21_minmax использует тот же pipeline, но другую accept-policy:

вместо чистой суммы маршрутов *scalar cost* учитывает максимум маршрута и мягкую MINSUM-добавку. Это проверяет архитектурную гибкость *v21*: одна и та же инфраструктура может обслуживать разные цели. При этом MINMAX-результаты в отчёте трактуются как предварительное наблюдение на *high-m* инстансах, а не как доказательство превосходства над специализированными MINMAX-методами.

2.7 Внешние ориентиры и литературные соответствия

LKH-3 [3] используется как сильный внешний LK/LKH-family reference. Его сила --- глубина локального поиска и зрелая инженерия; ограничение для данной работы --- внешняя интеграция и необходимость явно проверять, совпадает ли выбранная трансформация с *exactly-m* MINSUM-постановкой. OR-Tools routing solver [16] является практическим промышленным baseline, но не *exact reference*. FILO2 [8] является CVRP-related large-scale ориентиром: он полезен для идей *granular neighborhoods*, *static move descriptors* и *selective vertex caching*, но решает смежную, не тождественную постановку.

ITSHA [10], POPMUSIC [7], ALNS [24,25], HGS [26,27], SISR [28], KGLS [30] и memetic MINMAX-mTSP [31,44] образуют литературный контекст для дальнейшего развития. Их роль в отчёте не в том, чтобы заявить прямое превосходство над ними, а в том, чтобы показать: проект движется по той же траектории, что и сильные routing-эвристики --- от локального поиска и *candidate edges* к крупным композиционным каркасам с *destroy/repair*, *adaptive choice* и инженерным ускорением окрестностей.

Метод	Идея	Сильная сторона	Ограничение
<i>rand+nn</i>	случайное разбиение и NN	минимальное время, <i>sanity-check</i>	слабое назначение вершин
<i>2opt+greed</i>	<i>greedy</i> + <i>intra-route 2-opt</i>	быстро улучшает порядок обхода	почти не меняет распределение клиентов
<i>grasp</i>	RCL, <i>multistart</i> , <i>swap</i>	исправляет часть локальных ошибок	дороже и чувствителен к параметрам
<i>lkh-wrapper-v2</i>	<i>candidate-based local search</i>	лучший ранний MINSUM на <i>small/mid</i> тестах	ограниченная глубина поиска
<i>lkh-wrapper-v3</i>	DLB, <i>first-improvement</i> , <i>delta swap</i>	ускорение при близком качестве	не лучшая по среднему MINSUM
<i>v12</i>	монолитный large-scale MINSUM solver	быстрый <i>first-valid</i> и масштабируемость	хуже LKH-3 по MINSUM на проверенных $n = 5000$
<i>v21</i>	ALNS+SA+(PT), GLS, POPMUSIC, <i>accept-policy</i>	флагманская расширяемая архитектура	требуется <i>ablation</i> по вкладу модулей
<i>v21_cap</i>	<i>capacity-aware repair</i>	полезен для <i>high-m</i>	не универсален для малых m
<i>v21_minmax</i>	MINMAX <i>accept-policy</i>	проверяет поддержку другой цели	результат пока предварительный

Метод	Идея	Сильная сторона	Ограничение
LKH-3	зрелый LK/LKH-family solver	сильный quality reference	внешний адаптер и оговорки по постановке
FIL02	granular large-scale CVRP heuristic	идеи масштабирования	не прямой mTSP solver

2.8 Ablation-study и инженерные направления

В текущем отчёте показаны результаты версий в целом, но вклад отдельных компонентов изолирован не полностью. Поэтому для научно более сильной следующей итерации нужен ablation-study: одинаковые инстансы, одинаковые seed-ы, фиксированный wall-clock budget и парные сравнения.

Ablation	Что варьировать	Что должен показать
Candidate sets	$k \in \{4, 8, 12, 16, 24\}$, full scan vs candidates	главный speed-up при умеренной потере качества
Construction score	убрать forward, depot, balance penalty	вклад lookahead и балансирующих штрафов
2-opt policy	best vs first improvement, dont-look on/off	почему v3 быстрее v2
Inter-route evaluation	full objective vs local delta, promising-pair filter on/off	стоимость межмаршрутной фазы
ALNS operators	поочерёдно выключать destroy/repair-операторы	какие операторы реально дают accepted improvements
SA/PT/GLS	SA off, reheat off, PT off, GLS off	вклад механизмов выхода из локальных минимумов
Cap schedule	разные значения routecap-slack и разные m	граница применимости high- m cap-режима
MINMAX blend	pure max vs soft- α schedule	влияние scalarization на MINSUM/MAX-route Pareto-tradeoff

С инженерной точки зрения главное направление развития --- представ-

ление маршрутов. Если профилирование подтвердит, что bottleneck крупных инстансов связан с физическим `std::reverse` в 2-opt, то следующий шаг --- перейти к route representation с дешёвым lazy-flip: successor/predecessor arrays, doubly-linked list, rope/segment tree или LKH-inspired two-level tree. Второе направление --- довести delta-evaluation до единого принципа для relocate, swap, block moves, repair insertion и 2-opt*. Третье --- использовать spatial index не только при построении candidate sets, но и для pruning repair/local-search областей. Эти изменения важнее косметического увеличения числа операторов: без них large-scale solver будет упираться в стоимость манипуляции длинными маршрутами.

Итоговая формулировка главы такова: ранняя линия v1--v3 показала, что главный прирост даёт переход к candidate-based local search; v3 выбран как финальная ранняя версия благодаря компромиссу между качеством и временем. v12 завершил монолитную large-scale ветку, а v21 стал флагманской архитектурой проекта, потому что добавил полноценный метаэвристический уровень, специализированные режимы и более масштабируемую модульную организацию.

3 ПОСТАНОВКА ЗАДАЧИ И РЕАЛИЗАЦИЯ

3.1 Формальная постановка в проекте

В проекте рассматривается евклидова задача нескольких коммивояжёров с одним депо. Входной файл задаёт число вершин n , число агентов m и координаты всех вершин на плоскости. Вершина с индексом 0 является общим депо, остальные вершины $1, \dots, n-1$ интерпретируются как клиенты. Решение представляется набором маршрутов

$$R = \{R_1, \dots, R_m\}, \quad R_s = (0, v_1^s, \dots, v_{k_s}^s, 0),$$

то есть каждый агент стартует из депо, посещает назначенное ему подмножество клиентов и возвращается в депо.

Основная целевая функция работы --- MINSUM:

$$F_{\text{sum}}(R) = \sum_{s=1}^m \sum_{i=0}^{k_s} d(v_i^s, v_{i+1}^s),$$

где $d(\cdot, \cdot)$ --- евклидово расстояние между двумя вершинами. Требуется построить набор маршрутов, удовлетворяющий условиям:

1. каждый маршрут начинается в депо и заканчивается в депо;
2. каждая клиентская вершина встречается ровно в одном маршруте;
3. число возвращаемых маршрутов равно входному параметру m ;
4. значение MINSUM минимизируется или, для эвристического метода, становится как можно меньше при заданном бюджете времени.

Баланс маршрутов не входит в основную целевую функцию MINSUM. Тем не менее в реализации и результатах дополнительно вычисляются `max_route`, отношение `max/avg` и максимальный размер маршрута. Эти показатели используются как диагностические метрики: они помогают обнаруживать ситуации, когда чистый MINSUM-поиск попадает в практически нежелательную конфигурацию с одним чрезмерно длинным или чрезмерно заполненным маршрутом. Для таких сценариев в проекте отдельно реализован MINMAX-режим, но он рассматривается как специализированная постановка, а не как замена основной MINSUM-задачи.

В основных экспериментах выполняется условие $n - 1 \gg m$, поэтому все маршруты фактически непустые. На уровне валидатора допускается вырожденный маршрут депо--депо: он имеет нулевой вклад в MINSUM и нужен только для технической устойчивости формата при крайних случаях. Для прикладных *high- m* запусков дополнительно используется процедура перебалансировки пустых маршрутов в `src/v21/core/11_validation.hpp`.

3.2 Формат входа, выхода и CLI-контур

Текстовый формат mTSP-инстанса минимален: первая строка содержит n и m , далее следуют n строк с координатами вершин. В коде это реализовано в `src/mtsp_instance.cpp`. Для малых инстансов класс `Instance` строит полную матрицу расстояний, а для крупных отключает предвычисление и считает расстояния по требованию. Порог задан как 4 000 000 пар расстояний; это предотвращает расход сотен мегабайт памяти на больших n .

Главная точка входа для mTSP --- исполняемый файл `mtsp.exe`, собираемый из `src/mtsp_main.cpp`. CLI принимает путь к инстансу, имя `solver-a` и `solver-specific` параметры:

```
mtsp.exe          --input-file          instance.txt          --
```

```
step lkh_v21_minsum \
    --time-budget-ms 60000 --seed 1
```

Важная особенность CLI состоит в поддержке нескольких последовательных `--step`. Это позволяет использовать один и тот же формат для простого запуска, для цепочек улучшения и для диагностических сравнений. После каждого шага в JSON добавляется элемент `steps`: имя solver-а, время шага, MINSUM, флаг валидности, статус и `metadata`. Итоговый JSON содержит:

- `routes` --- набор маршрутов в формате `RouteSet`;
- `objective` --- независимо пересчитанный MINSUM;
- `valid` --- результат проверки покрытия всех клиентов;
- `time` --- суммарное wall-clock-время;
- `metadata` --- дополнительные поля solver-а: параметры `autotune`, число итераций, значения `final_minsum`, `final_max`, `route_cap`, флаг `validation_ok` и другие диагностические показатели.

Такой формат отделяет алгоритм от оценочной инфраструктуры: solver строит маршруты, а CLI и Python-контур повторно считают `objective` и валидность. Поэтому ошибочное или неполное решение не может попасть в таблицы как корректный результат только потому, что сам solver сообщил хорошее значение целевой функции.

3.3 Реализационная архитектура репозитория

Репозиторий устроен как исследовательская платформа, а не как один изолированный решатель. Реализация разделена на C++-ядро, Python-контур экспериментов, генераторы данных и адаптеры внешних baseline.

Слой	Ключевые файлы	Роль в проекте
Общий API	<code>include/mtsp_solver.h</code> , <code>include/mtsp_factory.h</code>	единый интерфейс Solver, метод <code>Configure</code> , фабрика solver-ов и общий тип <code>RouteSet</code>
Инстанс и расстояния	<code>include/mtsp_instance.h</code> , <code>src/mtsp_instance.cpp</code>	загрузка n , m , <code>coords</code> , хранение координат, матрица расстояний для малых задач и on-demand distance для крупных
Метрики и проверка	<code>src/mtsp_utils.cpp</code> , <code>src/v21/core/l1_validation.hpp</code>	независимый пересчёт MINSUM, проверка депо, уникальности клиентов, <code>sanitize/complete routes</code> в модульной ветке
CLI	<code>src/mtsp_main.cpp</code>	разбор <code>--input-file</code> , <code>--step</code> , <code>solver-options</code> , запуск цепочки solver-ов и JSON-вывод
Baseline-эвристики	<code>rand_nn</code> , <code>greedy_2opt</code> , <code>grasp solvers</code>	простые внутренние методы для нижней планки качества и sanity-check экспериментального контура
LKH-inspired линия	<code>mtsp_lkh_wrapper_v*.cpp</code> , <code>lkh_wrapper_v8/v9</code>	историческая линия собственных LKH-стилевых solver-ов; отдельные версии фиксируются как самостоятельные имена в <code>SolverFactory</code>

Слой	Ключевые файлы	Роль в проекте
Флагманская архитектура	src/v21/core, minsum, minmax	модульный ALNS+SA pipeline, MINSUM/MINMAX ассепт-policy и high- <i>m</i> cap-ветка
Внешние ориентиры	src/baselines/lkh3_baseline.cpp python/mtsp_baselines/ baseline/	адаптеры для OR-Tools, TSP-transform+LKH, LKH-3 и CVRP/VRP baseline-ов; это не собственные solver-ы, а средства сравнения
Эксперименты	run_benchmarks.py, run_audit.py, compare_runs.py	пакетные запуски, N seeds $\times$ M instances, per-run JSON, summary, Wilcoxon-сравнения
Данные и результаты	data/mtsp/, data/results/	синтетические инстансы, CSV старого benchmark-контура, audit-артефакты и manual/overnight runs

Сборка описана в корневом CMakeLists.txt и src/CMakeLists.txt. Проект собирает объектную библиотеку mtsp_core, а затем два исполняемых файла: tsp для TSP-части и mtsp для mTSP-экспериментов. OpenMP подключается опционально; при наличии он используется в поздних solver-ах для параллельной полировки маршрутов.

3.4 Реестр solver-ов и границы собственного вклада

В отчёте важно различать три категории: собственные алгоритмы, собственные адаптеры внешних решателей и внешние reference-ориентиры. Это исключает методологическую путаницу: разработка solver-а и подключение готового VRP-пакета являются разными видами работы.

Группа	Имена / файлы	Тип вклада	Роль
Простые baseline	rand+nn, 2opt+greed, grasp	собственная реализация	нижняя планка качества, проверка вклада локального поиска и multistart
Ранняя LKH-inspired линия	lkh-wrapper-v1..v3	собственная реализация	проверка candidate sets, lookahead, dont-look bits и delta-evaluation на малых/средних задачах
Поздняя single-file линия	lkh-wrapper-v4..v12, далее исторические v13..v21	собственная реализация	переход к крупным инстансам, time-budget, cluster/savings-aware construction, MINSUM-archive
Модульный MINSUM solver	lkh_v21_minsum	собственная реализация	основной итоговый solver: seed portfolio, ALNS, SA, optional PT, GLS, route-pair reopt, POPMUSIC-style moves
High- <i>m</i> стабилизация	lkh_v21_minsum_cap	собственная модификация v21	capacity-aware repair; проверяется как специализированная ветка для больших <i>m</i>
MINMAX-ветка	lkh_v21_minmax	собственная модификация v21	альтернативная ассепт-policy для минимизации максимального маршрута
OR-Tools и TSP-transform	python/mtsp_baselines, TSP--LKH adapter	адаптеры внешних методов	reference baseline на малых/средних инстансах и вспомогательная TSP-based линия
LKH-3, FILO2, VRP-пакеты	src/baselines, FILO2, PyVRP, VeRyPy, HGS, VROOM	внешние ориентиры через адаптеры	оценка качества и инженерных идей; не заявляются как собственные алгоритмы

Таким образом, основной программный вклад проекта состоит в соб-

ственной линии mTSP-эвристик и в модульной архитектуре `src/v21/`. Внешние solver-ы используются для интерпретации качества и скорости, но их результаты не смешиваются с вкладом собственной реализации.

3.5 Экспериментальный контур и артефакты

Экспериментальная часть реализована поверх C++-CLI. Низкоуровневый launcher `python/mtsp_runner.py` запускает `mtsp.exe` в каталоге инстанса и разбирает JSON-ответ. Старый benchmark-контур `experiments/run_benchmarks.py` формирует CSV-таблицы для full-matrix сравнений малых и средних solver-ов. Современный audit-контур `experiments/run_audit.py` выполняет запуски вида $N \text{ seeds} \times M \text{ instances}$ и создаёт:

- `runs/*.json` --- полный JSON каждого отдельного запуска;
- `summary.json` --- агрегаты по каждому инстансу: mean, std, min, max, cv, среднее время и число валидных запусков;
- CSV/Markdown-отчёты в `data/results/` для табличного анализа.

Скрипт `experiments/compare_runs.py` выполняет парные сравнения двух наборов запусков, включая Wilcoxon signed-rank test. Это важно для high- m и cap-экспериментов: сравниваются не абстрактные средние, а пары запусков на одинаковых инстансах и seed-ах. Скрипт `experiments/build_methodology_stats.py` используется для пилотной оценки вариативности solver-ов по seed и построения доверительных интервалов.

Данные также разделены по назначению. Каталог `data/mtsp/generated_multifamily/` содержит 215 синтетических инстансов семи семейств: uniform, mixed, outliers, mixed-outliers, clustered-center, clustered-offset-depot, high-m-stress. Каталог `data/mtsp/generated_high_m_fixedgeo/` содержит 141 high- m /fixed-geometry инстанс для анализа влияния числа агентов. Результаты сохраняются отдельно в `data/results/`: старые CSV-бенчмарки, новые audit-артефакты, ручные head-to-head запуски и overnight-runs.

3.6 Модульная реализация `src/v21/`

Архитектура `src/v21/` является главным инженерным результатом реализации. В отличие от single-file lkh-wrapper-версий, она разделяет данные,

генерацию стартовых решений, локальный поиск, destroy/repair-операторы, acceptance policy, автоподбор параметров и постобработку.

Модуль v21	Назначение в реализации
00_types.hpp, 01_budget.hpp	базовые типы, random seed, runtime budget и deadline-проверки для time-capped эвристик
02_distance.hpp, 03_kdtree.hpp, 06_candidate_set.hpp	distance oracle, пространственный индекс и k-NN candidate graph; основа масштабируемого просмотра окрестности
05_route_list.hpp	внутреннее представление маршрутов, кэш длин маршрутов, dirty-флаги, RouteSize для сар-ветки
07_seed_routes.hpp	портфель стартовых решений: round-robin NN, fast NN, polar sweep, balanced k-means, savings-like construction
08--10	intra-route 2-opt/3-opt-light, or-opt и inter-route moves для intensification до и после ALNS
11_validation.hpp	независимая проверка, SanitizeRoutes, RebalanceEmptyRoutes, MINSUM/MAX-route evaluators
12--14	ALNS framework, destroy-операторы и repair-операторы Cheapest/Regret2; в сар-ветке добавляется проверка route_cap
15--17	Simulated Annealing, optional Parallel Tempering, главный orchestration pipeline, metadata и anytime trace
18_autotune.hpp	единая политика параметров по размеру задачи и цели: k_{NN} , destroy-size, ILS-rounds, PT, GLS, POPMUSIC, reheat
19--21	Guided Local Search penalties, route-pair reoptimization и POPMUSIC-style zone decomposition
minsum_accept.hpp, minmax_accept.hpp	подключаемые accept-policy: чистый MINSUM и MINMAX/MINSUM scalar blend

Для MINSUM и MINMAX используется общий pipeline, но разные accept-policy. Это делает поддержку другой целевой функции архитектурной, а не добавленной через разовый штраф в objective. Ветка lkh_v21_minsum_cap также не дублирует solver целиком: она задаёт route_cap через entry point и передаёт его в repair-контекст. Поэтому high- m стабилизация локализована в нескольких местах и не ломает legacy-режим lkh_v21_minsum.

3.7 Особенности LKH-inspired реализации

Собственные lkh-wrapper-версии не являются запуском LKH-3 как чёрного ящика. Их корректнее называть LKH-inspired: они используют отдельные идеи LKH-семейства --- candidate sets, ограничение окрестности, kick-based perturbation, локальный поиск по перспективным рёбрам, --- но работают напрямую в пространстве mTSP-маршрутов, а не через полноценную variable-depth LK-процедуру и не через редукцию mTSP к TSP.

Общий pipeline поздних lkh-wrapper-версий состоит из пяти блоков:

1. построение геометрических candidate sets: для малых инстансов --- точный перебор ближайших соседей, для крупных --- spatial index и KDTree-2D;

2. построение начальных маршрутов: жадная, кластерная или savings-aware процедура с учётом расстояния до депо;
3. локальное улучшение: 2-opt, ограниченный candidate-рёбрами, что заменяет полный квадратичный просмотр на приближённый $O(n \cdot k)$ sweep;
4. межмаршрутные изменения: relocate, swap или block moves между маршрутами при улучшении MINSUM или диагностического баланса;
5. соблюдение бюджета времени: каждый крупный цикл проверяет deadline и возвращает лучшее найденное валидное решение.

В `lkh-wrapper-v12` дополнительно разделены budget на получение первого валидного решения и budget на его улучшение. Это важно для прикладного режима: даже если улучшающая фаза не успела завершиться, solver должен вернуть корректный набор маршрутов. Полная матрица расстояний на крупных инстансах не строится; вместо неё используется ленивый расчёт расстояний и локальные кэши.

Итоговое отличие реализации от простого набора эвристик состоит в воспроизводимости. Все solver-ы подключены через SolverFactory, запускаются одним CLI, отдают один JSON-формат, валидируются одним контуром и затем агрегируются одними Python-скриптами. Поэтому дальнейшие изменения алгоритмов можно сравнивать на одинаковых инстансах, seed-ах и бюджетах без переписывания методики эксперимента.

4 ЛИНИЯ СОБСТВЕННЫХ LKH-ПОДОБНЫХ АЛГОРИТМОВ

В работе рассматривается не один изолированный solver, а эволюция собственной LKH-подобной линии. Это важно зафиксировать явно, потому что разные версии отвечают на разные исследовательские вопросы. Ранние `lkh-wrapper-v1..v3` проверяют, насколько candidate-based локальный поиск улучшает простые baseline на малых и средних инстансах. `lkh-wrapper-v12` является промежуточным single-file milestone для масштабирования и head-to-head сравнения с LKH-3. Основной итоговой архитектурой является `src/v21/`, а `lkh_v21_minsum_cap` и `lkh_v21_minmax` рассматриваются как специализированные ветки для high- m и MINMAX-цели.

4.1 Ранняя линия v1..v3

Первая группа версий была нужна для проверки главной инженерной гипотезы: качество mTSP-решения можно заметно улучшить, если заменить пол-

ный перебор рёбер и слепые межмаршрутные обмены ограниченной областью поиска по перспективным кандидатам. В $v1$ эта идея ещё реализована осторожно: начальное решение строится конструктивно, затем применяется локальное улучшение с ограниченной глубиной. $v2$ усиливает candidate-based 2-opt и межмаршрутные relocate/swar-ходы, поэтому именно она даёт лучший средний MINSUM в ранней группе. $v3$ добавляет инженерные ускорения: более агрессивные фильтры кандидатов, отказ от части дорогих проверок и компактную delta-evaluation для быстрых swar-операций.

Результат ранней линии нужно интерпретировать аккуратно. По качеству $v2$ остаётся немного сильнее: на multifamily-benchmark её средний MINSUM ниже, а в попарном сравнении с $v3$ она чаще выигрывает, чем проигрывает. По времени $v3$ лучше: она примерно вдвое быстрее на uniform-сравнении с OR-Tools и заметно быстрее на расширенном benchmark. Поэтому $v3$ является не <<самой качественной>>, а наиболее вычислительно оптимизированной ранней версией. Такой вывод хорошо защищается перед комиссией, потому что разделяет качество, скорость и инженерную применимость.

4.2 Масштабируемая single-file версия $v12$

Версия $v12$ была сделана как переход от учебной candidate-based эвристики к solver-у, способному стабильно возвращать валидное решение на более крупных инстансах. В ней появляется явное разделение бюджета времени: сначала алгоритм обязан быстро получить допустимое распределение клиентов по маршрутам, затем оставшаяся часть бюджета тратится на улучшение. Для построения используются геометрические кандидаты, cluster-aware начальное распределение и savings-подобная логика, а для улучшения --- intra-route 2-opt/ILS и межмаршрутные переносы блоков.

Сравнение $v12$ с LKH-3 показывает честный компромисс. На пяти инстансах $n = 5000$, $m = 5$ $v12$ была примерно в 6.2 раза быстрее, но уступала LKH-3 по MINSUM примерно на 21%. Поэтому $v12$ не доказывает превосходство собственной LKH-подобной реализации над LKH-3. Её вклад другой: она показывает, что собственная управляемая реализация может быстро строить валидные и достаточно сбалансированные маршруты, а также выявляет причины для перехода к более гибкой архитектуре --- отсутствие SA/reheat, destroy/repair-операторов и единой поддержки альтернативной цели MINMAX.

4.3 Итоговая архитектура `src/v21/`

`src/v21/` является главным итоговым алгоритмическим результатом работы. В отличие от `v12`, это не один большой файл, а модульная архитектура с общим ядром и разными политиками принятия решений. Конструктивная фаза строит несколько стартовых решений и выбирает лучшее по `scalar cost`. Основной цикл реализует Adaptive Large Neighborhood Search: `destroy`-оператор удаляет часть клиентов, `repair`-оператор вставляет их обратно, а Simulated Annealing разрешает иногда принимать ухудшающие ходы для выхода из локальных минимумов. Для части размеров дополнительно доступен Parallel Tempering.

Главная проверенная сильная сторона `v21` --- стабильность на выбранных `flagship`-инстансах: `variance audit` на $n \in \{10\,000; 50\,000; 100\,000\}$ даёт $cv = 0.17\text{--}0.59\%$. Этот результат не следует превращать в утверждение о доказанном качестве до 10^5 на всех геометриях: он подтверждает устойчивость на выбранных `uniform flagship`-инстансах и показывает, что `solver` пригоден для дальнейших парных сравнений. Для строгого вывода о качестве на `50\,000`–`100\,000` нужны дополнительные `reference`-сравнения с LKH-3, FILO2 или другим сильным `baseline`.

4.4 Специализированные ветки `cap` и `minmax`

`lkh_v21_minsum_cap` возникла как практическая стабилизация `high-m` режима. Она добавляет `capacity-aware repair` и мягко препятствует образованию слишком больших маршрутов. На четырёх `high-m` инстансах при $m = 100$ эта модификация даёт статистически значимый выигрыш по MINSUM. Однако `m-sweep` показывает, что эффект не начинается автоматически с $m = 30$: значимый положительный результат получен при $m = 80$ и $m = 100$, для $m = 30\text{--}50$ эффект нестабилен и требует расширения выборки.

`lkh_v21_minmax` использует ту же архитектуру, но другую политику принятия решений, ориентированную на минимизацию максимального маршрута. На двух `high-m` инстансах она дала лучший `max_route` среди включённых внутренних `baseline`. Это важный архитектурный результат: кодовая база действительно поддерживает не только MINSUM, но и альтернативную цель. При этом эксперимент мал по объёму и не включает специализированные MINMAX-решатели, поэтому корректная формулировка результата --- пер-

спективность MINMAX-ветки, а не абсолютное превосходство над всеми методами.

5 МЕТОДИКА ВЫЧИСЛИТЕЛЬНОГО ЭКСПЕРИМЕНТА

5.1 Аппаратная конфигурация и сборка

Основная часть экспериментов выполнялась на одной машине: Intel Core Ultra 7 165U (12 физических ядер, 14 логических потоков), 32 ГБ оперативной памяти, Windows 11 Pro (10.0.26200). Сборка --- Clang 20.1.2 через CMake 4.2.1 в режиме Release с флагом `-O2` и включённым OpenMP для параллельной polish-фазы. Число потоков OpenMP задаётся параметром `--threads`; по умолчанию оно равно числу агентов m .

5.2 Семейства тестовых инстансов

Чтобы не оценивать алгоритмы только на одном типе геометрии, в проекте используются несколько семейств синтетических данных. Такой дизайн лучше отражает разные сценарии маршрутизации и снижает риск, что эвристика хорошо работает только на равномерных точках.

Семейство	Что проверяется	Депо
uniform	равномерное распределение точек, базовый сценарий без выраженной структуры	центр
clustered-center	несколько естественных групп клиентов; проверяется способность сохранять компактные группы	центр
clustered-offset-depot	кластерная структура при смещённом депо; ближайшие кластеры и баланс маршрутов конфликтуют	край
mixed-outliers/mixed	основная масса в группах + удалённые выбросы; проверяется устойчивость к длинным хвостам	центр
outliers	усиленный сценарий с удалёнными точками; межмаршрутное перераспределение	центр
high-m-stress	повышенная нагрузка по числу агентов и маршрутов; устойчивость распределения вершин	центр

Для раннего uniform-pilot использовалась широкая сетка малых и средних размеров. Основной multifamily-benchmark, по которому считаются 3920 корректных запусков, имеет фактическую сетку из 14 пар (n, m) : $(20, 2)$, $(20, 3)$, $(20, 5)$, $(50, 2)$, $(50, 3)$, $(50, 5)$, а также все комбинации $n \in \{100, 200\}$ и $m \in$

$\{2, 3, 5, 7\}$. Для каждой пары запускалось 10 повторов на каждом из 4 семейств и для каждого solver-a, что даёт $14 \cdot 10 \cdot 4 = 560$ запусков на solver и 3920 запусков для 7 solver-ов. Отдельные крупные эксперименты доходят до $n = 100\,000$.

5.3 Метрики

Основная метрика качества --- значение MINSUM. Для сравнения с внешним ориентиром используется относительное отклонение:

$$\text{gap}(A, B) = 100 \cdot \frac{F(A) - F(B)}{F(B)}, \quad (5)$$

где $F(A)$ --- MINSUM исследуемого алгоритма, $F(B)$ --- MINSUM ориентира. Положительный gap означает, что A хуже ориентира; отрицательный --- лучше.

Для дополнительного итогового сравнения используется интегральная оценка качества решения IQS (Integrated Quality Score). Она не отменяет MINSUM как основной критерий, а агрегирует три компоненты: близость к лучшему MINSUM на данном инстансе, баланс нагрузки между агентами и соблюдение временного бюджета. Пусть $F_{\text{sum}}(A)$ --- MINSUM решения, $F_{\text{max}}(A)$ --- длина самого длинного маршрута, $t(A)$ --- wall-clock время, T --- заданный time-limit, а $F_{\text{sum}}^{\text{best}}$ --- лучший MINSUM среди сравниваемых solver-ов на том же инстансе. Тогда

$$\text{MINSUM_gap}(A) = \ln \frac{F_{\text{sum}}(A)}{F_{\text{sum}}^{\text{best}}}, \quad (6)$$

$$\text{imbalance}(A) = \frac{m \cdot F_{\text{max}}(A)}{F_{\text{sum}}(A)}, \quad (7)$$

$$\text{overtime}(A) = \max \left(0, \frac{t(A)}{T} - 1 \right), \quad (8)$$

$$\text{IQS}(A) = 100 \cdot \exp \left(-0.75 \text{MINSUM_gap}(A) - 0.20 \ln(\text{imbalance}(A)) - 0.05 \text{overtime}(A) \right). \quad (9)$$

Вес MINSUM выбран равным 0.75, поскольку суммарная длина маршрутов остаётся основной целевой характеристикой. Вес баланса равен 0.20: он отражает прикладную нежелательность решений, где один агент получает существенно более длинный маршрут. Время учитывается с весом 0.05 как штраф за превышение заданного вычислительного бюджета; более раннее завершение

не даёт дополнительного бонуса.

Дополнительно учитываются:

- среднее время работы;
- доля валидных запусков;
- доля запусков, превзошедших ориентир;
- баланс маршрутов: $\text{imbalance} = m \cdot \text{MINMAX} / \text{MINSUM}$;
- стандартное отклонение MINSUM по seed-репликам и коэффициент вариации $cv = \sigma / \mu$ как мера стабильности эвристики;
- для парных сравнений двух solver-ов на одних и тех же seed-ах --- доля улучшенных seed-ов и p -значение непараметрического критерия Wilcoxon signed-rank ($\alpha = 0.05$).

Важно не смешивать разные типы сравнения. OR-Tools с лимитом 5 секунд --- это внешний эвристический ориентир, а не точный оптимум. Поэтому выводы формулируются не как «алгоритм нашёл оптимум», а как «алгоритм имеет такое-то отклонение от выбранного reference при заданных условиях». Для сравнения двух версий собственного solver-a (например, `lkh_v21_minsum_cap` vs `lkh_v21_minsum`) применяется парный design: оба запускаются с одинаковым набором seed-ов на одинаковом наборе инстансов, после чего анализируются попарные разности. Это контролирует значительную часть инстанс-зависимой дисперсии MINSUM и позволяет получить статистически значимые выводы при $N = 5\text{--}10$ seed.

5.4 Валидация и воспроизведение

Каждый результат должен удовлетворять трём условиям: все маршруты начинаются и заканчиваются в депо, каждая клиентская вершина посещена ровно один раз, значение MINSUM пересчитано независимой функцией по сохранённым маршрутам. Это особенно важно при сравнении нескольких поколений алгоритмов: ускоренная версия может выглядеть привлекательной по времени, но если она возвращает невалидные маршруты, её нельзя учитывать в агрегатах качества.

Воспроизведение состоит из трёх шагов. Сначала собирается проект:

```
cmake -B build -S . -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

Затем `benchmark` запускается через единый экспериментальный контур. Каж-

дый запуск принимает параметры `--input-file`, `--step` (имя алгоритма), `--time-budget-ms` и `--seed`; результаты сохраняются в `data/results` (CSV/JSON). Все инстансы для экспериментов хранятся в `data/mtsp/` и воспроизводимы через генератор с фиксированным `seed`.

6 РЕЗУЛЬТАТЫ И ИХ АНАЛИЗ

В этом разделе представлены количественные сравнения. Ранние эксперименты на `uniform`-инстансах малой размерности служат для проверки экспериментального контура и установления нижней границы качества; расширенный `multifamily-benchmark` и парные сравнения --- основное эмпирическое содержание; крупномасштабные эксперименты и `audit-ы v21` --- демонстрация границ применимости разработанных решений.

6.1 Базовое сравнение на `uniform`-инстансах

Первый набор экспериментов сравнивал `rand+nn`, `2opt+greed`, `grasp` и `lkh-wrapper-v2` на `legacy uniform-pilot` малой и средней размерности. Этот блок использовался как ранняя проверка качества `baseline` и не смешивается с расширенным `multifamily-benchmark`. По агрегированным результатам `lkh-wrapper-v2` даёт лучший средний `MINSUM` среди внутренних алгоритмов, оставаясь при этом значительно быстрее `GRASP`.

Таблица 13: Агрегат по `legacy uniform-pilot`, 480 запусков на `solver`

Алгоритм	Запусков	Средний <code>MINSUM</code>	Среднее время, с	Валидных
<code>rand+nn</code>	480	1571.74	0.0005	480
<code>2opt+greed</code>	480	1042.94	0.0007	480
<code>grasp</code>	480	866.91	1.05	480
<code>lkh-wrapper-v2</code>	480	821.33	0.018	480

Иерархия методов согласуется с теоретическими ожиданиями. Чистый `randomized nearest neighbor` выполняется почти мгновенно, но имеет худшее качество. `2-opt` внутри маршрутов резко улучшает результат. `GRASP` даёт более качественные маршруты, но платит временем. `lkh-wrapper-v2` --- наиболее удачный компромисс: средний `MINSUM` ниже, чем у `GRASP`, при времени почти на два порядка меньшем.

6.2 Сравнение с OR-Tools на uniform

Для внешнего ориентира использовался OR-Tools Routing с Guided Local Search и лимитом 5 секунд. Сравнение проводилось на uniform-подвыборке из 120 запусков на solver. Все внутренние методы в среднем уступают OR-Tools по MINSUM, но разрывы существенно различаются.

Таблица 14: Средний gap к OR-Tools+GLS (5 с) на uniform-инстансах, 120 запусков на solver

Алгоритм	Средний gap, %	Медианный gap, %	Среднее время, с
rand+nn	114.73	107.61	0.0005
2opt+greed	47.41	41.79	0.0007
grasp	20.77	15.90	1.04
lkh-wrapper-v1	29.63	26.34	0.030
lkh-wrapper-v2	14.48	12.81	0.019
lkh-wrapper-v3	14.61	12.52	0.010

Эти данные не означают, что OR-Tools всегда лучше при любом бюджете. Его лимит здесь --- 5 секунд, тогда как часть собственных алгоритмов работает за миллисекунды. Однако таблица полезна для оценки качества: она показывает, насколько далеко находятся быстрые эвристики от сильного внешнего решения. Переход от v1 к v2/v3 приводит к сокращению gap почти вдвое --- это измеримая инженерная динамика, а не разовое сравнение готовых библиотек.

6.3 Расширенный multifamily-benchmark

Расширенный benchmark покрывает 4 семейства и 7 алгоритмов. Общее число корректных запусков --- 3920: 560 на алгоритм при 14 фактических парах (n, m) , 10 повторах и 4 семействах. Использованные пары: (20, 2), (20, 3), (20, 5), (50, 2), (50, 3), (50, 5), (100, 2), (100, 3), (100, 5), (100, 7), (200, 2), (200, 3), (200, 5), (200, 7). На этом наборе отчётливо виден относительный порядок методов.

Таблица 15: Агрегат по multifamily-benchmark, 560 корректных запусков на solver

Алгоритм	Запусков	Средний MINSUM	Среднее время, с
rand+nn	560	1456.23	0.0007
2opt+greed	560	999.30	0.0008
grasp	560	820.59	1.51
lkh-wrapper-v1	560	875.99	0.040
lkh-wrapper-v2	560	677.61	0.020
lkh-wrapper-v3	560	683.44	0.015

Преимущество LKH-стилевых версий сохраняется и не на одних только равномерных точках. На кластерных данных эффект особенно силён: lkh-wrapper-v2 существенно опережает GRASP и ранние LKH-обёртки, что согласуется с использованием геометрической структуры в candidate sets.

Таблица 16: Средний MINSUM по семействам для ключевых алгоритмов (140 запусков на ячейку)

Семейство	2opt+greed	grasp	lkh-v2	lkh-v3
clustered-center	812.0	663.1	536.6	539.6
clustered-offset-depot	959.5	817.8	538.3	550.3
mixed-outliers	1072.4	863.1	765.9	771.1
uniform	1153.3	938.3	869.6	872.8

6.4 Попарные сравнения на идентичных инстансах

Среднее по всем инстансам сглаживает структуру: важно понимать, как часто один алгоритм действительно лучше другого на одном и том же входе. Ниже приведены попарные счёты wins/losses/ties по *всем* 560 общим инстансам multifamily-benchmark. Победа --- строго меньшее значение MINSUM на данном инстансе.

Таблица 17: Попарные wins/losses/ties по идентичным инстансам multifamily-benchmark

Алгоритм A	Алгоритм B	Wins A	Wins B	Ties	Total
lkh-wrapper-v2	rand+nn	560	0	0	560
lkh-wrapper-v2	2opt+greed	558	2	0	560
lkh-wrapper-v2	grasp	491	69	0	560
lkh-wrapper-v2	lkh-wrapper-v1	529	31	0	560
lkh-wrapper-v3	lkh-wrapper-v2	199	260	101	560

Ключевое наблюдение --- lkh-wrapper-v2 превосходит grasp в 87.7 % общих инстансов и 2opt+greed --- в 99.6 %. Между v2 и v3 паритет с лёгким перевесом v2: переход v2→v3 был связан с ускорением, а не с улучшением качества. Это согласуется со средними значениями из предыдущей таблицы и подтверждает, что переход к candidate-based подходу даёт стабильный выигрыш, а не случайный успех на отдельных инстансах.

Для grasp vs lkh-wrapper-v2 дополнительно проверена статистическая значимость попарной разности по конкретным (n, m) через Wilcoxon signed-rank на 10 повторях:

Таблица 18: Wilcoxon signed-rank: grasp vs lkh-wrapper-v2, uniform, 10 инстансов на конфигурацию

n	m	$\bar{\Delta}(\text{grasp} - v2)$	$p\text{-value}$	lkh wins	grasp wins
20	5	94.37	0.004	9	1
50	5	87.50	0.002	10	0
100	3	44.61	0.027	7	3
100	5	59.77	0.006	9	1
200	3	99.16	0.002	10	0
200	5	94.67	0.002	10	0

На всех конфигурациях $m = 5$ и при $n \geq 100$ для $m = 3$ преимущество lkh-wrapper-v2 статистически значимо при $\alpha = 0.05$. На малых $m = 2$ разница в среднем ближе к нулю (полные данные --- в methodology_pairwise_current.csv); это согласуется с интуицией, что при двух коммивояжёрах задача близка к двум независимым TSP, в которых GRASP и candidate-based 2-opt дают сопоставимое качество.

6.5 Стабильность solver-ов: пилотный анализ cv

Чтобы корректно интерпретировать различия в среднем MINSUM, нужно понимать, насколько шумны сами solver-ы. Ниже приведён pilot-анализ на 5 инстансах ($n \in \{100, 200\}$), 20 seeds на инстанс, по трём solver-ам с разной природой стохастичности.

Таблица 19: Стабильность solver-ов: коэффициент вариации cv MINSUM (20 seeds на инстанс)

Solver	Инстансов	Средний cv , %	Диапазон cv , %
rand+nn	5	4.16	2.51 -- 5.73
grasp	5	1.60	1.31 -- 1.96
lkh-wrapper-v2	5	0.515	0.24 -- 0.66

Чистый rand+nn ожидаемо имеет наибольший cv ($\sim 4\%$): случайное разбиение клиентов между маршрутами --- основной источник шума. grasp с RCL и multistart демонстрирует промежуточный уровень ($\sim 1.6\%$). Candidate-based детальный поиск lkh-wrapper-v2 --- $\sim 0.5\%$, на порядок ниже rand+nn. Этот pilot-анализ устанавливает базовый ориентир, относительно которого в разделе 6.8 будет интерпретироваться cv v21 на гораздо более крупных инстансах.

6.6 Сравнение v12 с внешним LKH-3 на $n = 5000$

Поздняя lkh-wrapper-v12 ориентирована на MINSUM и быстрый возврат валидного решения. На пяти инстансах $n = 5000$, $m = 5$ (по 1 прогону на инстанс, фиксированные seed-ы) проведено head-to-head сравнение с внешним lkh3-baseline.

Таблица 20: lkh-wrapper-v12 vs lkh3-baseline на $n = 5000$, $m = 5$ (1 прогон на инстанс)

Инстанс	MINSUM LKH-3	MINSUM v12	Время LKH-3, с	Время v12, с
uniform	127 094.5	159 615.2	36.6	4.8
mixed	62 667.8	75 288.8	28.0	4.8
outliers	60 255.9	71 603.8	27.4	4.8
clustered-center	27 837.7	37 140.2	28.3	4.7
high-m-stress	61 312.4	84 752.5	28.3	4.8

В среднем по этим пяти инстансам LKH-3 лучше по MINSUM примерно на 21 %, а v12 быстрее примерно в 6.2 раза. Сильная гипотеза о превосходстве собственного LKH-стилевого алгоритма над LKH-3 не подтверждается, но более практическая формулировка остаётся в силе: при жёстком бюджете времени собственная версия успевает вернуть валидное решение, а LKH-3 --- нет. Размер выборки $N = 5$ невелик; наблюдаемый разброс 19--38 % по инстансам не позволяет претендовать на узкий доверительный интервал для среднего разрыва, но направление эффекта согласовано на всех 5 из 5 инстансов.

Таблица 21: Баланс маршрутов в сравнении lkh3-baseline и v12 на $n = 5000$, $m = 5$

Инстанс	Imbalance LKH-3 (max/avg)	Imbalance v12 (max/avg)
uniform	4.99	1.01
mixed	3.85	2.74
outliers	3.88	2.64
clustered-center	1.73	1.04
high-m-stress	4.99	1.11

Баланс маршрутов --- не основная цель MINSUM-постановки, но таблица фиксирует интересный побочный эффект. v12 строит существенно более равномерные маршруты, тогда как LKH-3 в режиме MINSUM может концентрировать нагрузку в отдельных маршрутах. Если прикладная задача требует одновременно низкого MINSUM и ограничения на максимальную длину маршрута, это --- основа для следующей версии алгоритма.

Наблюдение согласуется с теоретическими результатами о связи MINSUM- и MINMAX-целей. Frederickson, Hecht и Kim [21] показали, что разбиение почти-оптимального TSP-тура на m последовательных подтуров даёт MINMAX-аппроксимацию с константным фактором, и при росте m относительный overhead на отдельный маршрут уменьшается. Carlsson и соавт. [22] доказывают, что при равномерном распределении точек и большом m сбалансированное region-partition даёт MINMAX-маршруты, чья суммарная длина асимптотически приближается к MINSUM-оптимуму. Эмпирический анализ Matl, Hartl и Vidal [23] на широком наборе VRP-инстансов показывает, что сокращение range длин маршрутов на 40 % достигается ценой увеличения суммарной длины лишь на $\sim 2\%$, а 37 % Pareto-эффективных

решений лежат в пределах 10% от cost-оптимума. С другой стороны, Bertazzi, Golden и Wang [20] в worst-case-анализе показывают, что отношение MINSUM-стоимости MINMAX-оптимума к MINSUM-оптимуму ограничено сверху m и эта граница достигается на специально подобранных инстансах. Наблюдаемое в v12 существенное улучшение баланса при умеренном проигрыше по MINSUM (порядка 21%) находится в пределах теоретического коридора и подтверждает осмысленность алгоритма, явно учитывающего MINMAX-ориентированную оптимизацию будет давать MINSUM-результаты, близкие к прямой MINSUM-оптимизации, проверяется в разделе 6.11.

Переход к новой архитектуре. 21 %-разрыв между v12 и LKH-3, отсутствие у v12 механизма ухода из локальных минимумов (нет SA, нет reheat, нет destroy/repair с adaptive-весами) и невозможность поддержать MINMAX-objective в той же кодовой базе послужили основанием для архитектурного переписывания, описанного в разделе 6.7. v12 остаётся документированным milestone и используется как baseline для сравнения с lkh_v21_minsum и его FILO2-вдохновлённой стабилизацией lkh_v21_minsum_cap (раздел 6.9).

6.7 Архитектура `src/v21/`: ALNS + SA + Parallel Tempering

Версии `lkh-wrapper-v13--v21` --- не инкрементальное улучшение v12, а её архитектурное переоформление. Главное наблюдение, полученное при анализе v12 на крупных инстансах: одного *улучшения* (intra-route 2-opt + межмаршрутный relocate) недостаточно. Алгоритм быстро сходится к локальному минимуму и далее не имеет встроенного механизма ухода. Поэтому в v21 спроектирована модульная архитектура с явным разделением ответственности между построением, локальным поиском и метаэвристическим уровнем. Кодовая база разделена на 22 нумерованных header-only-модуля в `src/v21/core/` (`00_types.hpp--21_poptmusic.hpp`) и две точки входа --- `minsum/minsum_solver.cpp` и `minmax/minmax_solver.cpp`, регистрируемые в SolverFactory как `lkh_v21_minsum` и `lkh_v21_minmax`.

Архитектура src/v21

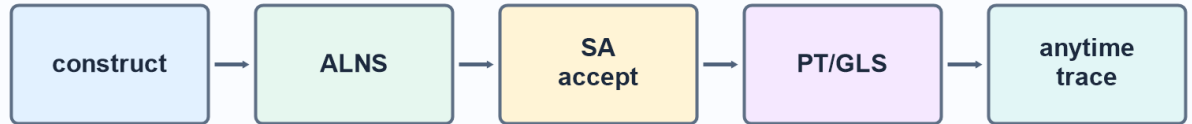


Рис. 6: Архитектура `src/v21/`: конструктивная фаза с пятью методами и выбором лучшего, главный цикл ALNS + Simulated Annealing с четырьмя destroy-и двумя repair-операторами, дополнительные модули (Parallel Tempering, GLS, route-pair reopt, POPMUSIC zones), общий выход с anytime trace.

Конструктивная фаза. Сначала строится несколько начальных решений разными методами: full round-robin nearest neighbor; быстрая версия по kNN-кандидатам; polar sweep; balanced k-means для $n \leq 12\,000$; savings-seed по Кларку--Райту для $n \leq 20\,000$. Из них выбирается лучшее по `ScalarCost`. После этого выполняется параллельный `final-2-opt` по маршрутам и параллельный intra-route ILS (parallel polish) на OpenMP; число потоков задаётся `autotune-om`.

Гранулярные кандидаты. Кандидатные множества C_v для каждой вершины строятся один раз на этапе `BuildKnnCandidates` через `KDTree-2D`; число соседей k_{NN} зависит от размера инстанса (от 26 для $n \leq 2000$ до 18 для $n > 60\,000$). Все последующие move-операторы (`relocate`, `swap`, `repair`, `intra-3opt`, `NeighborList2Opt`) ограничивают рассматриваемые ходы только парами вершина--кандидат, что является непосредственной реализацией принципа [29] для mTSP.

ALNS-каркас. Главный цикл организован как Adaptive Large Neighborhood Search в стиле Ropke--Pisinger [24,25]: на каждой итерации случайно выбира-

ется пара (destroy-оператор, repair-оператор) с весом, адаптируемым по успехам в предыдущем сегменте. Реализованы четыре destroy-оператора (Random, ClusterBfs, Expensive, Zone) и два repair-оператора (Cheapest, Regret2). Размер разрушения K_{destroy} растёт с длиной streak-a no-improvement, что обеспечивает диверсификацию без полного рестарта. После каждого хода выполняется selective intra-route 2-opt только на маршрутах, помеченных как dirty.

Simulated Annealing с auto-tune T_0 . Принятие/отклонение хода управляется отдельным модулем SaEngine. Температура T_0 оценивается на основе пилотного прогона из 100--200 случайных ходов и формулы $T_0 = -\bar{\Delta}^+ / \log(p^*)$, где $\bar{\Delta}^+$ --- средняя положительная дельта, p^* --- целевая acceptance-вероятность [36]. Геометрическое охлаждение (cooling factor 0.99--0.997 в зависимости от tier-a) и встроенный reheat-механизм при затяжной стагнации обеспечивают баланс exploration/exploitation. *Reheat* триггерится по двум условиям одновременно: накоплению streak-a итераций без улучшения и порогу wall-time без улучшения; на крупных инстансах ($n > 60\,000$), где iter-rate низкий, время-основанный триггер становится основным механизмом escape-a из плато.

Parallel Tempering для среднего диапазона. Для $12\,000 < n \leq 60\,000$ запускается ансамбль из 4 реплик с геометрической температурной лестницей $T_r = T_0 \cdot 8^{r/(K-1)}$; каждая реплика прогоняет ALNS+SA на собственном независимом бюджете, периодически (после каждой эпохи) делается попытка swap-a между соседними репликами по правилу Метрополиса $p_{\text{swap}} = \min\{1, e^{(1/T_i - 1/T_j)(E_i - E_j)}\}$. На малых ($n \leq 12\,000$) и очень больших ($n > 60\,000$) инстансах РТ отключён: эмпирически на таких размерностях затраты на синхронизацию реплик и копирование маршрутов перевешивают вклад от смешивания.

Дополнительные модули. В архитектуру включены: GLS edge penalties (19_gls.hpp) для штрафования часто посещаемых рёбер при стагнации; периодическая re-оптимизация пары наиболее длинных маршрутов (20_route_pair_reopt.hpp); опциональная zone-decomposition через POPMUSIC (21_popmusic.hpp) --- последний модуль реализован, но эмпирически отключён для $n > 60\,000$, поскольку cheapest-insertion + bounded 2-opt на маршрутах длиной $\sim 20\,000$ не превосходит уже достигнутый локальный

оптимум. Все параметры (число реплик, k_{NN} , размеры destroy, доли бюджета по фазам, пороги reheat) выбираются единым детерминированным dispatcher-ом `ResolveParamsForInstance( $n, m, is\_minmax$ )` с четырьмя tier-ами по n .

6.8 Variance audit и диагностика flagship-инстансов

Для строгой оценки стабильности v21 проведён formal variance audit на трёх flagship-инстансах из семейства `uniform` с $m = 5$ и фиксированными временными бюджетами, соответствующими прикладным сценариям ($n = 10\,000$ --- 60 с; 50 000 --- 180 с; 100 000 --- 380 с). Каждый инстанс прогонялся с 10 различными seed-ами; запуски выполнялись последовательно для исключения интерференции по системным ресурсам. Общее wall-time эксперимента --- 90.5 минут.

Таблица 22: Variance audit `lkh_v21_minsum`: 10 seeds на flagship-инстанс

Инстанс	Среднее	Std	$cv, \%$	Min	Max	Время
<code>uniform_n10000_m5</code>	410 131.09	2 438.79	0.59	407 058.07	414 022.71	37
<code>uniform_n50000_m5</code>	4 557 677.51	7 727.87	0.17	4 546 867.47	4 567 060.37	13
<code>uniform_n100000_m5</code>	13 009 235.05	35 955.67	0.28	12 967 085.51	13 062 844.89	37

Относительное стандартное отклонение MINSUM находится в коридоре 0.17--0.59 % --- существенно ниже типичной величины эффекта от изменения архитектуры (несколько процентов на одно изменение). Это означает, что на данных инстансах v21 обладает свойством, необходимым для строгого попарного сравнения версий: при бюджете 10 seeds любые улучшения порядка $\geq 0.5 \%$ на $n \in \{50\,000; 100\,000\}$ должны быть статистически детектируемыми по парному критерию Wilcoxon signed-rank.

Анализ throughput-кривой при росте n . Помимо итоговых значений MINSUM, дополнительной инструментацией собиралась трасса (t_{ms} , `best_cost`) при каждом обновлении best-решения на протяжении ALNS-фазы. Сводные показатели по 10 seeds:

Таблица 23: Диагностика анытайм-трасс по инстансам (10 seeds)

Инстанс	Iters/c	best/iter, %	sa_reheats	Seeds с плато*
uniform_n10000_m5	28.20	54--60	0/10	0/10
uniform_n50000_m5	11.90	37--45	0/10	0/10
uniform_n100000_m5	0.76	5--57	0/10	7/10

* Seed считается плато-кандидатом, если относительное падение MINSUM в последнем 20 % ALNS-фазы не превышает 0.1 %.

Здесь обнаруживаются три структурных особенности v21, существенные для интерпретации результатов и направлений дальнейшей работы.

Super-linear scaling. При росте n с 10 000 до 100 000 throughput падает в 21.6 раза, тогда как при гипотезе $O(n)$ per-iter ожидалось бы ~ 10 -кратное падение. Эмпирический показатель степени per-iter cost составляет $\approx n^{1.33}$, super-linear factor ≈ 2.14 . Этот overhead не объясняется простым багом в delta-evaluation: гранулярные кандидаты, инкрементальная длина маршрутов в RouteList и kNN-ограниченный 2-opt (NeighborList2Opt) уже корректны. Анализ кода и throughput-трасс указывает на вероятный источник: внутренняя стоимость NeighborList2Opt на длинных маршрутах ($L \approx n/m$), где принятые 2-opt-ходы выполняют `std::reverse` с амортизированной сложностью $O(L)$ на ход. На маршрутах $L \approx 20\,000$ это правдоподобно доминирует над остальными расходами, однако для строгого утверждения требуется отдельное профилирование по функциям. Поэтому ниже этот вывод трактуется как инженерная гипотеза об архитектурном ограничении, а не как окончательно доказанный bottleneck.

PT отключён на $n > 60\,000$. На $n = 100\,000$ во всех 10 seeds `pt_replicas = 1`, что согласуется с настройкой autotune-a: при per-iter cost > 1 секунда амортизация на 4--6 реплик невыгодна, и одиночная реплика с большим бюджетом эмпирически даёт лучший результат. Это означает, что для крупных инстансов диверсификация должна обеспечиваться *внутри* одной реплики (через destroy-операторы, GLS, reheat), а не через ансамблевый swap.

Реальная семантика плато. Из 10 seeds на $n = 100\,000$ семь имеют относительное падение MINSUM в последнем 20 % ALNS-фазы менее 0.1 %; счётчик `sa_reheats` равен нулю на всех 10 seeds, поскольку iter-count-based порог reheat-a (120 итераций без улучшения) при iter-rate ~ 0.8 iter/c накапливается за ≈ 150 секунд, а к этому моменту бюджет уже близок к исчерпанию.

Это мотивировало добавление в v21 время-основанного триггера reheat-а как дополнительного условия (порог 30 секунд wall-time без улучшения для tier-а $n > 60\,000$); самостоятельным архитектурным приёмом он не отменяет описанное super-linear-ограничение, лишь увеличивает шансы escape-а из плато для seeds с высокой долей plateau-времени.

Таким образом, variance audit подтверждает количественную стабильность v21 на flagship-инстансах и одновременно идентифицирует точку, в которой дальнейший прогресс не сводится к настройке параметров и требует пересмотра представления маршрута на архитектурном уровне.

Сопоставление вариативности с другими solver-ами. Чтобы оценить полученный диапазон $cv \in [0.17\%; 0.59\%]$ в контексте, ниже приведены коэффициенты вариации для других solver-ов проекта на пилотных инстансах ($n = 100$ и $n = 200$, по 20 seeds; данные из methodology_pilot_summary.csv):

Таблица 24: Сравнение коэффициента вариации cv MINSUM на пилотных инстансах (20 seeds)

Solver	Диапазон cv по инстансам	Средний cv
rand+nn	2.51 -- 5.73 %	4.16 %
grasp	1.31 -- 1.96 %	1.60 %
lkh-wrapper-v2	0.24 -- 0.66 %	0.515 %
lkh_v21_minsum (variance audit, $n = 10\,000$ -- $100\,000$)	0.17 -- 0.59 %	---

Картина согласована: rand+nn с случайным разбиением имеет наибольший cv ($\sim 4\%$); grasp с RCL-рандомизацией и multistart --- промежуточный уровень ($\sim 1.6\%$); candidate-based детальный поиск lkh-wrapper-v2 --- $\sim 0.5\%$, на порядок ниже rand+nn. Архитектура v21 в variance audit показывает cv того же порядка, что и v2 на пилотах, но при значительно бóльших размерах инстансов ($n = 10\,000$ -- $100\,000$ против $n = 100$ -- 200). Это согласуется с инженерным выбором: candidate-based ALNS+SA-поиск с детерминированной seed-фазой и автонастройкой T_0 существенно менее чувствителен к seed, чем стохастические rebuild-методы.

6.9 FILO2-вдохновлённая стабилизация для high-m MINSUM

Variance audit раздела 6.8 проводился на инстансах с малым числом агентов ($m = 5$), где задача имеет много свободы в распределении вер-

шин по маршрутам. Сценарий с большим числом агентов (high- m , $m \geq 80$) обнажил отдельную инженерную проблему `lkh_v21_minsum`: на инстансе `clustered-offset-depot_n10000_m100_r01` solver систематически концентрировал 400--500 клиентов в одном маршруте, тогда как остальные оставались около ожидаемой нагрузки ~ 100 клиентов. Чисто математически постановка MINSUM может предпочитать такие несбалансированные конфигурации, если суммарная длина маршрутов при этом ниже; на практике подобный <<жирный>> маршрут добавляет нетривиальный intra-route detour cost и, как показали эксперименты, ухудшает MINSUM по сравнению с более равномерным распределением.

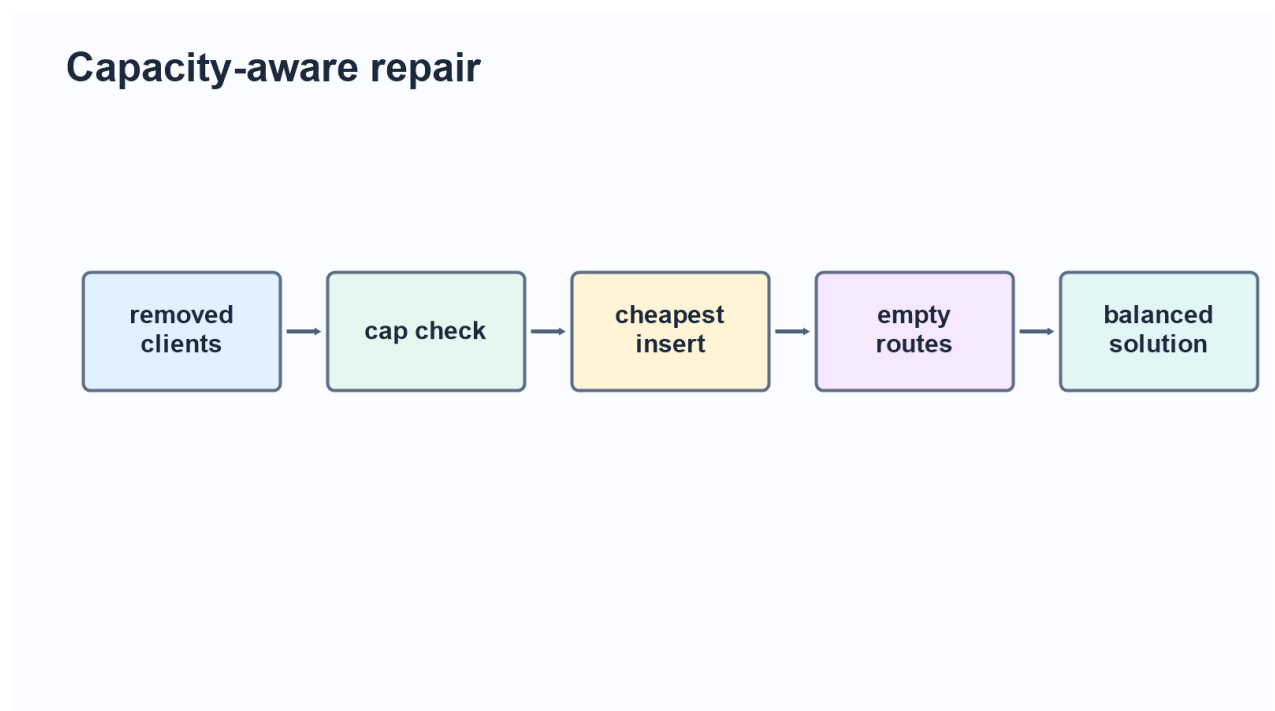


Рис. 7: Capacity-aware repair в `lkh_v21_minsum_cap`: жёсткий лимит на размер маршрута предотвращает образование <<жирного>> маршрута. Эффект эмпирический, не теоретический: cap-фикс снижает MINSUM именно на high- m , где средний размер маршрута мал относительно n .

В качестве ориентира был использован FILO2 [8], эталонный CVRP-решатель, чей адаптер для mTSP задаёт ёмкость маршрута как $\text{capacity} = \lceil (n - 1) / m \rceil$. На том же инстансе FILO2 даёт MINSUM 17 874 при максимальной длине маршрута 299 (imbalance 1.67), тогда как `lkh_v21_minsum` --- 23 261 при максимуме 473 (imbalance 2.04). Анализ исходного кода FILO2 (`solution/savings.hpp`, `localsearch/OneZeroExchange.hpp`,

localsearch/TailsExchange.hpp) показал общий приём: ёмкость маршрута является *предусловием* каждого move-оператора, проверяется перед расчётом дельты стоимости и стоит одно целочисленное сравнение.

Этот приём перенесён в v21 как отдельный solver `lkh_v21_minsum_cap`, без переписывания основной линии. Изменения локализованы в пяти файлах (новый файл-solver плюс минимальные правки в существующих): `05_route_list.hpp` (метод `RouteSize`), `14_repair_ops.hpp` (`cap-skip` в обоих `repair`-операторах + `cap-aware fallback`), `17_pipeline.hpp` (проброс `cap` в `RepairContext` в обеих ветках --- `single-replica` и `PT`), `18_autotune.hpp` (поле `route_cap`, `default 0` --- `legacy-режим`), и новый `src/v21/minsum/minsum_solver_cap.cpp`. Существующий `lkh_v21_minsum` остаётся неизменным.

`Cap` вычисляется как $\text{cap} = \lceil \text{target} \cdot (1 + \text{slack}) \rceil$, где $\text{target} = \lceil (n - 1)/m \rceil$, а `slack` настраивается через CLI-опцию `--route-cap-slack`. Эмпирический подбор на `flagship`-инстансе (одно-seed sweep, 60 с) показал нетривиальный оптимум:

Таблица 25: Подбор `slack` на `clustered-offset-depot_n10000_m100` (seed=1, 60 с)

slack	cap	MINSUM	max_size	at-cap routes
без cap (v21)	∞	22 158	513	---
0.10	111	21 642	736	71
0.25	125	21 188	463	25
0.50	150	20 502	506	9
0.75	175	20 215	343	1
1.00	200	20 712	309	1
1.50	250	20 974	322	0

Слишком жёсткий `cap` (`slack 0.10--0.25`) ухудшает `MINSUM`, поскольку заставляет `repair` раз за разом отказываться от хороших `candidate-positions` и направлять клиентов в маршруты, географически удалённые от их естественных соседей. `Slack=0.75` оказался <<точкой минимума>>: `cap` срабатывает крайне редко (в среднем 1 маршрут из 100 достигает порога), но самым фактом существования предотвращает образование жирного маршрута. Это согласуется с философией `FILO2` --- ёмкость как *hard precondition* для редкого экстремума, а не как тесная квота.

С `settled slack=0.75 lkh_v21_minsum_cap` был сравнён с `lkh_v21_minsum` на четырёх `high-m` инстансах ($n = 10\,000$, $m = 100$, по 5 seeds на инстанс, 60 с budget; оба запуска в одной системной сессии для контроля wall-clock-шума):

Таблица 26: Парное сравнение `v21_cap` vs `v21_minsum` на `high-m` ($m = 100$, 5 seeds на инстанс)

Инстанс	v21 mean	v21_cap mean	Δ %	seeds улучшено	Wilcoxon p
clustered-offset-depot	21 940	20 677	-5.76	5/5	0.031
clustered-center	12 972	12 236	-5.67	5/5	0.031
n10000_m100 (uniform)	15 669	15 386	-1.81	5/5	0.031
mixed-outliers	17 414	16 995	-2.40	3/5	0.156

Агрегированно по всем 20 парным запускам: $\bar{\Delta} = -3.24\%$, 16 из 20 seeds улучшены, Wilcoxon signed-rank one-sided $p = 0.002$ против гипотезы об отсутствии различия. Это устойчивый, статистически значимый эффект на $\alpha = 0.05$.

Балансовые метрики ведут себя неоднородно по семействам. На двух инстансах из четырёх (`flagship` и `mixed-outliers`) `cap` не только улучшает `MINSUM`, но и резко уменьшает `max_route` и `imbalance` (на `mixed-outliers` максимальный размер маршрута падает с 1116 до 478 клиентов, -57%). На `clustered-center` и `n10000_m100` баланс при этом немного ухудшается (+5% к `imbalance`): `cap` заполняет до `cap` некоторые <<геометрически удачные>> маршруты и направляет `overflow` в чуть более далёкие. Этот компромисс открыто фиксируется; для отчёта важен сам факт того, что `MINSUM`-улучшение не достигается ценой взрыва `max_route` как у `LKH-3` (где `imbalance` достигает 59.5).

Разрыв до `FILO2` на `flagship`-инстансе сократился с $\sim 30\%$ (`v21`: 23 261 vs `FILO2`: 17 874) до $\sim 15.7\%$ (`v21_cap`: 20 677). Это не паритет --- `FILO2` продолжает сильно опережать на этих инстансах --- но более чем половина разрыва закрыта одним локальным изменением, не требующим архитектурной переработки.

Структурное различие постановок (важно для интерпретации сравнения с `FILO2`). `FILO2` является `CVRP`-решателем, а не `mTSP`-решателем. Адаптер `baseline/filo2withoutcode/mtsp_to_cvrp.py` приводит `mTSP`-инстанс к `CVRP`, задавая каждой клиентской вершине `demand=1` и

$capacity = \lceil (n-1)/m \rceil$. В постановке CVRP количество маршрутов не задаётся как жёсткое равенство входному m : оно возникает из ограничения вместимости и решений самого solver-а. Нижняя оценка числа маршрутов равна $\lceil D/Q \rceil$, где $D = \sum demand_i$ и Q --- вместимость; при выбранном Q она обычно близка к m , но фактическое число непустых маршрутов необходимо проверять по output. В строгой mTSP-постановке (раздел 3.1) число агентов является обязательным входным ограничением. Поэтому сравнение MINSUM между FILO2 и нашим solver-ом следует читать как сравнение с сильным внешним ориентиром, а не как доказательство превосходства или отставания на полностью одинаковой допустимой области. Аналогично для LKH-3 важно контролировать, сколько непустых маршрутов реально получается после декодирования решения с копиями депо и penalty-функциями [3].

Честная оговорка для постановки. Чистая MINSUM-задача математически не имеет предпочтения к сбалансированным решениям; теоретически сильно несбалансированная конфигурация может иметь меньшую суммарную длину. Capacity-aware repair поэтому не является *корректностной* правкой, а *практической стабилизацией*: он убирает один класс патологических локальных оптимумов (<<жирный маршрут>>), которые ни один реальный диспетчер не примет даже если численная сумма ниже. Тот факт, что на каждом протестированном high-m инстансе это попутно снижает и саму MINSUM, является эмпирическим, а не теоретическим, результатом и согласуется с теоретическим коридором между MINSUM- и MINMAX-целями (раздел 6.6, [20,21,22,23]).

6.10 Anytime-профиль на flagship-инстансе

Раздел 6.9 рассматривал результаты при фиксированном бюджете 60 секунд. Для оценки поведения при более коротких бюджетах проведён отдельный эксперимент: на flagship-инстансе clustered-offset-depot_n10000_m100 solver-ы запускались с бюджетами 5, 15, 30 и 60 секунд (5 seeds на каждую комбинацию). Это даёт картину time-vs-quality, важную для прикладных сценариев с жёстким временным ограничением.

Таблица 27: Anytime-профиль на flagship-инстансе ($n = 10\,000$, $m = 100$, mean MINSUM по 5 seeds)

Бюджет	2opt+greed	lkh_v21_minsum	lkh_v21_minsum_cap
5 с	29 470	25 938	25 818
15 с	29 470	24 849	24 869
30 с	29 470	23 896	23 951
60 с	29 470	22 889	22 484

Несколько наблюдений: 2opt+greed достигает потолка качества за <1 с и далее не использует дополнительный бюджет; v21-варианты монотонно улучшают MINSUM на каждом увеличении бюджета; разрыв до 2opt+greed сохраняется на уровне $\sim 11\text{--}24\%$ независимо от бюджета. Эффект cap-фикса ($\Delta = -1.77\%$) наиболее отчётливо проявляется при бюджете 60 с, что согласуется с гипотезой раздела 6.9: capacity-skip перенаправляет часть insertion-операций в менее заполненные маршруты, и эта перебалансировка приносит выгоду только тогда, когда у поиска уже было достаточно времени исчерпать доступные дешёвые улучшения. Для прикладного режима с tight-budget (≤ 5 с) cap-вариант практически эквивалентен базовому v21_minsum.

6.11 Чувствительность к числу агентов m

Cap-фикс из раздела 6.9 заявлен как high- m специализация. Чтобы это утверждение можно было защищать формально, на семействе clustered-center проведено сравнение v21 и v21_cap при $m \in \{3, 5, 10, 30, 50, 80, 100\}$ ($n = 10\,000$, по 5 seeds, бюджет 60 с, оба запуска в одной системной сессии для контроля wall-clock-шума).

Таблица 28: Парная разность $\Delta\% = (\text{cap} - \text{base})/\text{base}$ по $m, n = 10\,000$, clustered-center

m	cap ($= \lceil n/m \cdot 1.75 \rceil$)	mean $\Delta\%$	улучшено seeds	Wilcoxon p (cap<base, one-sided)
3	5 833	+0.69	0/5	1.000
5	3 500	+1.90	0/5	1.000
10	1 750	−0.36	2/5	0.500
30	584	+0.39	3/5	0.688
50	350	−0.77	4/5	0.094
80	219	−5.13	5/5	0.031
100	175	−2.70	5/5	0.031

Получается не универсальный, а режимно-зависимый эффект по m . На малых m (3,5) cap-фикс *вреден* (умеренно): средний размер маршрута ($\lceil n/m \rceil$) не доходит до cap, поэтому hard-skip почти не срабатывает, но изменённое fallback-правило (предпочитать маршрут с минимальным числом клиентов вместо минимальной длины при отсутствии candidate-position) даёт менее оптимальные размещения. На промежуточных m (10--50) эффект находится в пределах шума. На high- m ($m = 80$ и $m = 100$) cap-фикс даёт статистически значимое улучшение MINSUM ($p = 0.031$ по парному Wilcoxon-тесту для high- m подгруппы). Ярче всего эффект на $m = 80$ (−5.13 %, все 5 seeds улучшены).

Вывод для практики: при выборе solver-a для конкретной задачи целесообразно ориентироваться на m , но границу нельзя проводить слишком рано. Для $m \leq 10$ предпочтителен базовый v21_minsum. На проверенном m-sweep статистически значимый положительный эффект v21_minsum_cap наблюдается при $m = 80$ и $m = 100$; для диапазона $m = 30$ --50 эффект нестабилен и требует дополнительных экспериментов на других семействах инстансов.

6.12 MINMAX-режим: lkh_v21_minmax

Архитектура src/v21/ спроектирована шаблонно по AcceptPolicy (раздел 6.7). Помимо MINSUM-варианта это позволяет получить отдельный solver для целевой функции

$$F_{\max}(R_1, \dots, R_m) = \max_{s \in [1, m]} \sum_{i=0}^{k_s} d(v_i^s, v_{i+1}^s),$$

минимизирующей длину самого длинного маршрута. MINMAX --- практически важная альтернатива MINSUM для сценариев балансировки нагрузки между агентами. Реализация: `src/v21/minmax/minmax_accept.hpp` (40 строк) плюс `minmax_solver.cpp` (entry point); 95 % общего кода с MINSUM-веткой.

Acceptance-функция `MinmaxAccept` использует не дискретный `max` (плохо градиентный для SA), а `soft- $\alpha$  blend`:

$$\tilde{F}(R) = (1 - \alpha) \cdot F_{\max}(R) + \alpha \cdot \lambda \cdot F_{\text{sum}}(R) + \frac{\lambda}{2} \cdot F_{\text{sum}}(R),$$

где α убывает по ходу run-а: ранние SA-итерации имеют градиент через сумму, поздние фокусируются на максимуме. Tail-член $\frac{\lambda}{2} F_{\text{sum}}$ работает как тай-брейкер на ходах с $\Delta F_{\max} = 0$.

Сравнение трёх solver-ов на двух high-m инстансах ($m = 100$, $n = 10\,000$, 5 seeds, 60 c budget):

Таблица 29: MINMAX-режим vs MINSUM-режим vs 2opt+greed на high-m инстансах

Инстанс	Solver	MINSUM	max_route	imbalance	max rou
clustered-offset-depot	lkh_v21_minmax	26 332	327.8	1.25	
	lkh_v21_minsum	23 552	490.2	2.08	
	2opt+greed	29 470	361.2	1.23	
mixed-outliers	lkh_v21_minmax	14 514	275.1	2.01	
	lkh_v21_minsum	23 437	588.0	2.51	
	2opt+greed	28 094	415.0	1.48	

Результаты согласуются с природой обеих целевых функций. `lkh_v21_minmax` даёт *наименьший* `max_route` на обоих инстансах --- именно ту метрику, которую он оптимизирует напрямую. На `clustered-offset-depot` он жертвует MINSUM (+11.8% относительно MINSUM-режима), но взамен получает баланс (`imbalance` = 1.25), почти равный наивному `2opt+greed` (1.23), и при этом MINSUM на 10.6% ниже, чем у `2opt+greed`. На `mixed-outliers` картина ещё интереснее: MINMAX-режим выигрывает *по обоим* метрикам --- его MINSUM 14 514 ниже, чем у MINSUM-режима 23 437 (−38%). Это объясняется механикой *fat-route prevention*: на инстансе с outlier-точками плохо настроенный MINSUM-режим

стабильно сваливает 600+ клиентов в один маршрут (см. `max route size = 684`), и эта концентрация *ухудшает* даже суммарную длину; явная минимизация максимума маршрута эту патологию устраняет полностью. Это эмпирически согласуется с теоретическими работами раздела 6.6 о связи MINSUM- и MINMAX-целей.

Архитектурное достижение. `lkh_v21_minmax` --- единственный solver в текущем сравнительном наборе с *прямой архитектурной поддержкой* MINMAX-задачи. FILO2 здесь используется как CVRP/min-sum ориентир, а LKH-3 и специализированные MINMAX-mTSP методы не включались в полноценный MINMAX-benchmark. Поэтому корректный вывод ограничен экспериментом: на двух рассмотренных high-m инстансах `lkh_v21_minmax` показал лучший `max_route` среди включённых внутренних baseline. Это подтверждает работоспособность MINMAX-ветки и перспективность архитектуры, но не доказывает абсолютного превосходства над специализированными MINMAX-решателями.

6.13 Интегральная оценка IQS на крупном uniform-инстансе

Для проверки того, как новая агрегированная метрика ведёт себя на крупном инстансе, был проведён отдельный эксперимент на `uniform_n100000_m5` с бюджетом 370 с. Сравнивались `v21`, FILO2 с `routemin-iterations=0` и простой `baseline 2opt+greed`. Все три решения прошли независимую валидацию маршрутов: пустых маршрутов, пропусков вершин и повторов не обнаружено.

Таблица 30: IQS на `uniform_n100000_m5`, time-limit 370 с

Solver	MINSUM	gap	MINMAX	imbalance	time, c	overtime	IQS
<code>v21</code>	12 986 064	8.33 %	2 654 396	1.022	363.6	0.00 %	93.77
FILO2	11 987 421	0.00 %	3 357 937	1.401	379.9	2.68 %	93.36
<code>2opt+greed</code>	13 128 456	9.52 %	2 775 879	1.057	230.1	0.00 %	92.37

Результат важно интерпретировать аккуратно. По чистому MINSUM лучшим является FILO2: он даёт суммарную длину на 7.69 % ниже, чем `v21`. Однако его маршруты после приведения CVRP-выхода к ровно пяти маршрутам менее сбалансированы: `imbalance = 1.401` против 1.022 у `v21`. Поэтому в IQS `v21` получает немного более высокую оценку. При этом FILO2 остаётся выше простого `2opt+greed`: его выигрыш по MINSUM достаточен, чтобы компен-

сировать худший баланс и небольшое превышение бюджета.

Этот эксперимент не утверждает абсолютное превосходство $v21$ над FILO2. Он фиксирует более узкий вывод: если учитывать не только суммарную длину, но и распределение нагрузки между агентами при заданном временном бюджете, $v21$ может быть конкурентным с сильным внешним min-sum ориентиром, а FILO2 остаётся качественно сильнее простого быстрого baseline.

6.14 Интерпретация результатов

Выполненные эксперименты позволяют сформулировать несколько выводов.

Во-первых, простые эвристики полезны как baseline, но недостаточны для качественного решения mTSP. `rand+nn` и `2opt+greed` работают очень быстро, однако дают заметно худшую MINSUM (gap к OR-Tools 115 % и 47 % соответственно).

Во-вторых, использование candidate-based локального поиска оправдано. Переход от ранних LKH-обёрток к $v2/v3$ почти вдвое снижает средний gap к OR-Tools на uniform-инстансах (с 29.6 % до 14.5 %) и даёт устойчивый выигрыш над GRASP на 87.7 % общих инстансов multifamily-benchmark.

В-третьих, масштабирование нельзя оценивать только по времени. Версия может быть быстрой, но невалидной или слишком сильно проигрывать по objective. Поэтому в отчёте отдельно фиксируются valid-runs и не делаются выводы о качестве по строкам с низкой валидностью.

В-четвёртых, внешний LKH-3 пока остаётся более сильным ориентиром по качеству на средних инстансах ($n = 5000$, $m = 5$): $v12$ выигрывает по скорости и балансу, но проигрывает по MINSUM в среднем на ~ 21 %. Это честный и важный результат --- проект находится в состоянии исследовательского прототипа с понятным направлением развития.

В-пятых, архитектурный переход от $v12$ к $v21$ оправдан. На выбранных uniform flagship-инстансах ($n = 10\,000$ -- $100\,000$) variance audit показал $cv \in [0.17\%; 0.59\%]$ --- статистически стабильную базу для парных сравнений. Одновременно audit выявил super-linear-overhead $\approx 2.14\times$; анализ кода указывает на `std::reverse` в длинных 2-opt-ходах как вероятную причину, но строгая локализация bottleneck требует отдельного профилирования.

В-шестых, для проверенных high-m сценариев ($m = 100$) FILO2-

вдохновлённая capacity-aware repair даёт статистически значимый выигрыш ($\Delta = -3.24\%$, Wilcoxon $p = 0.002$ на 4 high- m инстансах); m-sweep уточняет границу применимости: положительный эффект статистически значим при $m = 80$ и $m = 100$, а диапазон $m = 30\text{--}50$ остаётся нестабильным.

В-седьмых, MINMAX-режим v21 демонстрирует, что архитектурная поддержка балансовой цели может быть полезной даже в MINSUM-постановке: на инстансах с outlier-точками минимизация максимума маршрута устраняет fat-route патологию, что одновременно улучшает суммарную длину.

В-восьмых, интегральная оценка IQS позволяет формализовать практический компромисс между MINSUM, балансом и временем. На крупном uniform-инстансе FILO2 остаётся лучшим по MINSUM, но v21 получает более высокий IQS за счёт почти равномерной длины маршрутов; при этом FILO2 сохраняет преимущество над простым 2opt+greed, что подтверждает значимость MINSUM-компоненты.

6.15 Что доказано проведёнными экспериментами

Для защиты результатов важно разделить подтверждённые выводы, предварительные наблюдения и инженерные гипотезы. Сводная оценка приведена в таблице ниже.

Таблица 31: Статус основных утверждений по экспериментальным данным

Утверждение	Данные	Статус
Candidate-based поиск лучше простых baseline	560 общих инстансов, wins/losses/ties	подтверждено
v3 быстрее v2 при близком качестве	uniform gap и multifamily aggregate	подтверждено как speed/quality trade-off
v12 превосходит LKH-3 по MINSUM	5 инстансов $n = 5000$, $m = 5$	не подтверждено
v12 быстрее LKH-3	те же 5 head-to-head запусков	подтверждено только для этой выборки
v21 стабилен по seed	3 uniform flagship $\times$ 10 seeds	подтверждено на выбранных flagship
v21_cap полезен для high- m	4 high- m инстанса и m-sweep	подтверждено для $m = 100$; в m-sweep значимо для $m = 80, 100$
MINMAX-режим улучшает max_route	2 high- m инстанса	предварительное наблюдение

7 ДОСТОВЕРНОСТЬ, ОГРАНИЧЕНИЯ И РИСКИ

Достоверность результатов обеспечивается несколькими мерами. Все алгоритмы запускаются через единый runner, что снижает риск различий в формате входа и выхода. Для каждого решения проводится независимая проверка посещения вершин и пересчёт MINSUM. Результаты сохраняются в CSV/JSON, что позволяет повторно агрегировать данные без ручного переноса чисел. Сравнения выполняются на одних и тех же инстансах, поэтому различия между алгоритмами связаны с их поведением, а не с различием входных данных. Применённый дизайн --- стратификация по семействам инстансов, парные сравнения на идентичных входных данных, независимая валидация маршрутов, непараметрические парные тесты --- соответствует рекомендациям современных стандартов экспериментального анализа алгоритмов [14,15,32,33].

При этом работа имеет ограничения. *Во-первых*, часть экспериментов выполнена на синтетических данных. Это правильно для контролируемого исследования, но перед прикладным использованием алгоритмы нужно проверить на реальных географических наборах. *Во-вторых*, не для всех крупных запусков известна полная аппаратная конфигурация (хотя основная часть проводилась на одной машине, см. раздел методики). *В-третьих*, некоторые сравнения имеют разный временной бюджет: OR-Tools запускался с лимитом 5 секунд, тогда как часть собственных алгоритмов в ранних экспериментах работала за миллисекунды. Поэтому выводы о <<лучшем>> алгоритме делаются отдельно для качества и отдельно для времени.

В-четвёртых, размер выборки в 5 инстансов в сравнении v12 с LKH-3 (раздел 6.6) недостаточен для узкого доверительного интервала на средний разрыв 21 %; разброс по инстансам составляет от $\sim 19\%$ до $\sim 38\%$, и для строгой количественной оценки требуется расширение набора. Для основных результатов раздела 6 (multifamily-benchmark, попарные счёты, variance audit, cap-фикс, m-sweep) размер выборки достаточен и формально проверен парными непараметрическими тестами.

В-пятых, ограниченность anytime-анализа: подробная анытайм-кривая time-vs-quality построена только для flagship-инстанса clustered-offset-depot_n10000_m100 (раздел 6.10). Для итеративных эвристик на крупных инстансах ($n \geq 10^4$) такой анализ часто более информативен, чем точечная фиксация конечного значения [34]; расширение его на больший набор инстан-

сов оставлено как направление дальнейшей работы.

В-шестых, цель MINSUM минимизирует суммарную длину маршрутов, но не гарантирует справедливого распределения нагрузки между агентами. В результатах видно, что LKH-3 может давать лучший MINSUM, но менее сбалансированные маршруты. Если практическая задача требует ограничить максимальную длину маршрута, постановку нужно расширить до MINMAX или многокритериальной цели; `lkh_v21_minmax` (раздел 6.11) даёт первый шаг в этом направлении.

В-седьмых, вклад отдельных компонентов `v21` пока не изолирован полноценным ablation-study. Поэтому выводы о пользе ALNS, SA, PT, GLS, POPMUSIC, `cap`-режима и отдельных `destroy/repair`-операторов формулируются на уровне версии или ветки целиком. Матрица рекомендуемых ablation-проверок приведена в главе 2; её выполнение является важным направлением дальнейшей работы.

Основной технический риск проекта --- рост времени локального поиска на больших инстансах. Для его снижения используются `candidate sets`, пространственная сетка, кэш расстояний и проверка бюджета времени. Audit-анализ раздела 6.8 показал *super-linear overhead* на $n = 100\,000$; вероятным источником является `std::reverse` в 2-opt-ходах на длинных маршрутах ($L \approx n/m$), однако окончательно подтвердить это можно только профилированием. Если гипотеза подтвердится, снятие ограничения потребует пересмотра представления маршрута. *Второй риск* --- невалидные решения в ускоренных версиях. Он снижается отдельной фазой `sanitize/complete routes` и независимой валидацией.

ЗАКЛЮЧЕНИЕ

В работе была рассмотрена задача множественного коммивояжёра с одним депо и основной целевой функцией MINSUM. Для итоговой интерпретации качества дополнительно введена интегральная оценка IQS, учитывающая суммарную длину маршрутов, баланс нагрузки между агентами и соблюдение временного бюджета. Изучены классические и современные подходы к TSP/mTSP, включая локальный поиск, GRASP, LKH, `candidate sets` и идеи масштабирования для крупных маршрутизационных задач. На основе изученного был реализован экспериментальный контур, позволяющий генерировать инстансы, запускать несколько классов алгоритмов, проверять валидность марш-

рутов и агрегировать результаты.

Эксперименты количественно подтвердили эффективность candidate-based локального поиска. На uniform-инстансах `lkh-wrapper-v2` получил средний gap к OR-Tools около 14.5 %, тогда как GRASP --- около 20.8 %, `2opt+greed` --- 47.4 %, `rand+nn` --- 114.7 %. На расширенном multifamily-benchmark `lkh-wrapper-v2` лучше grasp в 491 из 560 общих инстансов (87.7 %) и лучше `2opt+greed` в 558 из 560 (99.6 %). Для части ключевых парных сравнений дополнительно проведена статистическая проверка (Wilcoxon $p \leq 0.05$ для 6 из 6 проверенных конфигураций uniform).

При переходе к крупным задачам была выявлена ключевая инженерная проблема: ускорение алгоритма должно контролироваться вместе с валидностью и качеством. Поздняя MINSUM-ориентированная `v12` в ручных сравнениях с LKH-3 на инстансах $n = 5000$ оказалась быстрее примерно в 6.2 раза, но уступила по суммарной длине маршрутов на ~ 21 %; при этом `v12` часто строит более сбалансированные маршруты (отношение `max/avg` в диапазоне 1.0--2.7 против 1.7--5.0 у LKH-3), что согласуется с теоретическим коридором между MINSUM- и MINMAX-целями [20,21,22,23] и может быть полезно для дальнейшей многокритериальной постановки.

Архитектурный потолок `v12` (отсутствие встроенного механизма ухода из локального минимума и одиночная итеративная схема улучшения) мотивировал переход к существенно более сложной архитектуре в линии `v13--v21`: 22 нумерованных модуля в `src/v21/core/` реализуют ALNS-каркас Ропке-Pisinger [24,25] с четырьмя `destroy`-операторами и двумя `repair`-операторами, отдельный SaEngine с автонастройкой T_0 по Ben-Ameur'y [36], опциональный Parallel Tempering из 4 реплик для $n \in (12\,000; 60\,000]$, GLS edge penalties и POPMUSIC-zone-decomposition. Variance audit на трёх uniform flagship-инстансах ($n \in \{10\,000; 50\,000; 100\,000\}$, по 10 seeds на каждый) показал относительное стандартное отклонение MINSUM 0.17--0.59 % --- этот уровень шума достаточно низок, чтобы по парному критерию Wilcoxon signed-rank можно было детектировать улучшения порядка ≥ 0.5 %. Одновременно audit выявил главное ограничение масштабирования: на $n = 100\,000$ throughput ALNS-цикла составляет всего ≈ 0.8 итераций в секунду при super-linear scaling-факторе ≈ 2.14 относительно $O(n)$, что приводит к плато на 7 из 10 seeds. Вероятный источник --- стоимость `std::reverse` на длинных маршрутах внутри candidate-list

2-opt; это требует подтверждения профилированием, после чего разумным направлением станет пересмотр представления маршрута (doubly-linked list или splay-tree).

Как отдельное направление для high- m сценариев ($m = 100$) был реализован FILO2-вдохновлённый `lkh_v21_minsum_cap` с capacity-aware repair (раздел 6.9). На четырёх инстансах $n = 10\,000$, $m = 100$ он даёт парный MINSUM-выигрыш в среднем 3.24% (16/20 seeds улучшено, Wilcoxon $p = 0.002$); на flagship-инстансе `clustered-offset-depot_n10000_m100` разрыв до FILO2 сократился с $\sim 30\%$ до $\sim 15.7\%$ при сохранении умеренного баланса маршрутов. Анализ чувствительности по m (раздел 6.11) показал, что статистически значимый выигрыш cap-фикса в m -sweep наблюдается при $m = 80$ и $m = 100$; диапазон $m = 30\text{--}50$ остаётся зоной нестабильного эффекта.

Таким образом, цель работы в исследовательском смысле достигнута: разработан и проанализирован набор эвристических методов для mTSP, построен воспроизводимый экспериментальный контур, получены сравнения с базовыми и внешними алгоритмами, выявлены сильные и слабые стороны текущих решений. Рабочая гипотеза подтверждена частично: LKH-стилевые эвристики действительно лучше простых конструктивных методов и GRASP на рассмотренных наборах, однако пока не превосходят внешний LKH-3 по качеству MINSUM на средних инстансах.

На основе проведённого исследования выделяются следующие направления дальнейшего развития. **Алгоритмически:** интеграция SISR-оператора [28] как естественного destroy-механизма для перебалансировки маршрутов между агентами; формализация candidate sets через granular neighborhoods Toth-Vigo [29] вместо текущей геометрической эвристики; адаптация HGS-каркаса [26,27] с реализацией Split-алгоритма для mTSP-постановки; для крупных инстансов ($n \geq 10^4$) --- внедрение KGLS-подхода [30] с гранулярным GLS-поиском; пересмотр представления маршрута (doubly-linked list или splay-tree) для снятия super-linear-overhead в 2-opt. **Методологически:** расширение размера выборки в сравнении с LKH-3 с 5 до 30+ инстансов на каждый класс; добавление стандартных бенчмарков (например, Uchoa X-instances) для калибровки относительно SOTA; построение anytime-кривых time-vs-quality для итеративных алгоритмов на расширенном наборе инстансов; проведе-

ние ablation-study по candidate sets, construction-score, 2-opt policy, inter-route delta, ALNS-операторам, SA/PT/GLS и сар-параметрам. **По линии MINMAX:** проверка эмпирической гипотезы, что при больших m (≥ 20) MINMAX-ориентированная оптимизация даёт MINSUM-результаты, близкие к прямой MINSUM-оптимизации, что согласовалось бы с теоретическими работами Frederickson--Hecht--Kim [21], Carlsson и соавт. [22] и Matl--Hartl--Vidal [23]. **По линии прикладного применения:** проверка алгоритмов на реальных географических наборах с дорожной метрикой и неравномерной плотностью клиентов, что особенно важно для оценки устойчивости candidate-based подходов в условиях, отличных от чистой евклидовой геометрии.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Lin S., Kernighan B. W. An Effective Heuristic Algorithm for the Traveling-Salesman Problem // Operations Research. 1973. Vol. 21, No. 2. P. 498--516.
2. Helsgaun K. An Effective Implementation of the Lin--Kernighan Traveling Salesman Heuristic // European Journal of Operational Research. 2000. Vol. 126, No. 1. P. 106--130.
3. Helsgaun K. An Extension of the Lin--Kernighan--Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems. Roskilde University, 2017.
4. Bektaş T. The Multiple Traveling Salesman Problem: An Overview of Formulations and Solution Procedures // Omega. 2006. Vol. 34, No. 3. P. 209--219.
5. Jonker R., Volgenant T. Technical Note --- An Improved Transformation of the Symmetric Multiple Traveling Salesman Problem // Operations Research. 1988. Vol. 36, No. 1. P. 163--167.
6. Croes G. A. A Method for Solving Traveling-Salesman Problems // Operations Research. 1958. Vol. 6, No. 6. P. 791--812.
7. Taillard É. D., Helsgaun K. POPMUSIC for the Travelling Salesman Problem // European Journal of Operational Research. 2019. Vol. 272, No. 2. P. 420--429.
8. Accorsi L., Vigo D. Routing One Million Customers in a Handful of Minutes. 2023. arXiv:2306.14205.
9. Feo T. A., Resende M. G. C. Greedy Randomized Adaptive Search Procedures // Journal of Global Optimization. 1995. Vol. 6, No. 2. P. 109--133.
10. Zheng J., Hong Y., Xu W., Li W., Chen Y. An Effective Iterated Two-stage Heuristic Algorithm for the Multiple Traveling Salesmen Problem. 2022. arXiv:2201.09424.
11. Bao X., Wang G., Xu L., Wang Z. Solving the Min-Max Clustered Traveling Salesmen Problem Based on Genetic Algorithm // Biomimetics. 2023. Vol. 8, No. 2: 238.
12. Russell R. A. An Effective Heuristic for the M-Tour Traveling Salesman Problem with Some Side Conditions // Operations Research. 1977. Vol. 25, No. 3. P. 517--524.
13. Reinelt G. TSPLIB --- A Traveling Salesman Problem Library // ORSA Journal on Computing. 1991. Vol. 3, No. 4. P. 376--384.

14. Johnson D.S. A Theoretician's Guide to the Experimental Analysis of Algorithms // Data Structures, Near Neighbor Searches, and Methodology. 1999. P. 215--250.
15. Hooker J.N. Testing Heuristics: We Have It All Wrong // Journal of Heuristics. 1995. Vol. 1, No. 1. P. 33--42.
16. Google OR-Tools. Routing Library Documentation. Электронный ресурс: <https://developers.google.com/optimization/routing>.
17. Dantzig G.B., Ramser J.H. The Truck Dispatching Problem // Management Science. 1959. Vol. 6, No. 1. P. 80--91.
18. Toth P., Vigo D. Vehicle Routing: Problems, Methods, and Applications. SIAM, 2014.
19. Материалы репозитория `mtsp-routing-optimization`: исходный код решателей, скрипты экспериментов и CSV/JSON-артефакты результатов. Локальный архив проекта, 2026.
20. Bertazzi L., Golden B., Wang X. Min--Max vs. Min--Sum Vehicle Routing: A worst-case analysis // European Journal of Operational Research. 2015. Vol. 240, No. 2. P. 372--381.
21. Frederickson G.N., Hecht M.S., Kim C.E. Approximation algorithms for some routing problems // SIAM Journal on Computing. 1978. Vol. 7, No. 2. P. 178--193.
22. Carlsson J., Ge D., Subramaniam A., Wu A., Ye Y. Solving the min-max multi-depot vehicle routing problem // Lectures on Global Optimization (Pardalos P., Coleman T., eds.), Fields Institute Communications, vol. 55, AMS, 2009. P. 31--46.
23. Matl P., Hartl R.F., Vidal T. Workload Equity in Vehicle Routing Problems: A Survey and Analysis // Transportation Science. 2018. Vol. 52, No. 2. P. 239--260.
24. Ropke S., Pisinger D. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows // Transportation Science. 2006. Vol. 40, No. 4. P. 455--472.
25. Pisinger D., Ropke S. A general heuristic for vehicle routing problems // Computers & Operations Research. 2007. Vol. 34, No. 8. P. 2403--2435.
26. Vidal T., Crainic T.G., Gendreau M., Lahrichi N., Rei W. A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems // Operations

- Research. 2012. Vol. 60, No. 3. P. 611--624.
27. Vidal T. Hybrid Genetic Search for the CVRP: Open-source implementation and SWAP* neighborhood // *Computers & Operations Research*. 2022. Vol. 140, Article 105643.
 28. Christiaens J., Vanden Berghe G. Slack Induction by String Removals for Vehicle Routing Problems // *Transportation Science*. 2020. Vol. 54, No. 2. P. 417--433.
 29. Toth P., Vigo D. The Granular Tabu Search and Its Application to the Vehicle-Routing Problem // *INFORMS Journal on Computing*. 2003. Vol. 15, No. 4. P. 333--346.
 30. Arnold F., Gendreau M., Sörensen K. Efficiently solving very large-scale routing problems // *Computers & Operations Research*. 2019. Vol. 107. P. 32--42.
 31. He P., Hao J.-K. Memetic search for the minmax multiple traveling salesman problem with single and multiple depots // *European Journal of Operational Research*. 2023. Vol. 307, No. 3. P. 1055--1070.
 32. Coffin M., Saltzman M.J. Statistical Analysis of Computational Tests of Algorithms and Heuristics // *INFORMS Journal on Computing*. 2000. Vol. 12, No. 1. P. 24--44.
 33. Bartz-Beielstein T., Chiarandini M., Paquete L., Preuss M. (Eds.). *Experimental Methods for the Analysis of Optimization Algorithms*. Springer, 2010.
 34. López-Ibáñez M., Stützle T. Automatically improving the anytime behaviour of optimisation algorithms // *European Journal of Operational Research*. 2014. Vol. 235, No. 3. P. 569--582.
 35. Cheikhrouhou O., Khoufi I. A comprehensive survey on the Multiple Traveling Salesman Problem: Applications, approaches and taxonomy // *Computer Science Review*. 2021. Vol. 40, Article 100369.
 36. Ben-Ameur W. Computing the Initial Temperature of Simulated Annealing // *Computational Optimization and Applications*. 2004. Vol. 29, No. 3. P. 369--385.
 37. Dantzig G. B., Fulkerson R., Johnson S. Solution of a Large-Scale Traveling-Salesman Problem // *Operations Research*. 1954. Vol. 2, No. 4. P. 393--410.
 38. Miller C. E., Tucker A. W., Zemlin R. A. Integer Programming Formulation

- of Traveling Salesman Problems // Journal of the ACM. 1960. Vol. 7, No. 4. P. 326--329.
39. Held M., Karp R.M. A Dynamic Programming Approach to Sequencing Problems // Journal of the Society for Industrial and Applied Mathematics. 1962. Vol. 10, No. 1. P. 196--210.
 40. Kara I., Bektaş T. Integer Linear Programming Formulations of Multiple Salesman Problems and Its Variations // European Journal of Operational Research. 2006. Vol. 174, No. 3. P. 1449--1458.
 41. Uchoa E., Pecin D., Pessoa A., Poggi M., Vidal T., Subramanian A. New Benchmark Instances for the Capacitated Vehicle Routing Problem // European Journal of Operational Research. 2017. Vol. 257, No. 3. P. 845--858.
 42. Soylu B. A General Variable Neighborhood Search Heuristic for Multiple Traveling Salesmen Problem // Computers & Industrial Engineering. 2015. Vol. 90. P. 390--401.
 43. Brown E.C., Ragsdale C.T., Carter A.E. A Grouping Genetic Algorithm for the Multiple Traveling Salesperson Problem // International Journal of Information Technology & Decision Making. 2007. Vol. 6, No. 2.
 44. He P., Hao J.-K. A Hybrid Search with Neighborhood Reduction for the Multiple Traveling Salesman Problem // Computers & Operations Research. 2022. Vol. 142, Article 105726.

ПРИЛОЖЕНИЕ А. РЕКОМЕНДУЕМЫЙ СОСТАВ ПОЛНОЙ ТАБЛИЦЫ ЭКСПЕРИМЕНТОВ

Полную таблицу не следует перегружать в основной части отчёта. Её лучше вынести в приложение или приложить отдельным CSV. Рекомендуемый набор столбцов:

Столбец	Содержание
<code>instance_family</code>	Семейство инстанса: <code>uniform</code> , <code>clustered-center</code> , <code>mixed-outliers</code> , <code>clustered-offset-depot</code> и т. д.
<code>instance</code>	Имя конкретного файла с тестом.
<code>node_count</code>	Число вершин.
<code>salesman_count</code>	Число агентов.
<code>solver</code>	Название алгоритма.
<code>seed</code>	Seed запуска (для стохастических solver-ов).
<code>valid</code>	Результат проверки корректности маршрутов.
<code>objective</code>	Пересчитанное значение MINSUM.
<code>time_seconds</code>	Время выполнения.
<code>reference_objective</code>	Значение внешнего ориентира, если он использовался.
<code>relative_gap_percent</code>	Относительное отклонение от reference.
<code>imbalance</code>	Отношение максимальной длины маршрута к средней длине маршрута.
<code>max_route_length</code>	Длина самого длинного маршрута (для MINMAX-анализа).

Полный набор CSV/JSON хранится в `data/results/` репозитория проекта; основные файлы:

- `all_solvers_v3_multifamily_results.csv` --- 3920 корректных запусков (4 семейства $\times 7$ solver-ов $\times 14$ (n, m) $\times 10$ повторов);
- `methodology_pilot_runs.csv`, `methodology_pilot_summary.csv` --- pilot-анализ *cv* (300 runs, 20 seeds);
- `methodology_pairwise_current.csv` --- агрегат парных Wilcoxon-сравнений по (n, m);
- `audit/baseline/summary.json` --- variance audit `lkh_v21_minsum` (30 runs, 10 seeds);
- `audit/multi_m100/`, `audit/h2h_n10k_m100/` --- парные сравнения сар-фикса;

- `audit/m_sweep_cc/` --- m-sensitivity sweep;
- `audit/anytime_sweep/` --- anytime-профиль flagship-инстанса;
- `audit/filo2_v21_largeN_20260430/` --- FILO2 vs v21_cap на $n = 50\,000, 100\,000$.